# 12. Optimal Control and Policy Based Reinforcement Learning

# 1. Optimal Control

- **Discrete-Time Optimal Control**
  - ✓ Dynamic Programming
  - ✓ Affine Quadratic Problem
  - ✓ Infinite Horizon Linear Quadratic Problem

- **Continuous-Time Optimal Control**
  - ✓ Dynamic Programming (Hamilton-Jacobi-Bellman Equation)
  - ✓ Minimum Principle
  - ✓ Affine Quadratic Problem
  - ✓ Infinite Horizon Linear Quadratic Problem

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | **Dynamic Optimization** | Dynamic Game |

**Action space**

| **Time space** | **Model based** | Finite | Infinite |
|---|---|---|---|
| | Discrete | Discrete time MDP $P(s_{t+1}\|s_t, a_t)$ | Discrete-time dynamic system $x_{t+1} = f(x_t, u_t)$ |
| | Continuous | Continuous time MDP $P(s_{t+h}\|s_t, a_t)$ | Continuous-time dynamic system $\dot{x}_t = f(x_t, u_t)$ |

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | **Dynamic Optimization** | Dynamic Game |

**Action space**

| **Time space** | **Model free** | Finite | Infinite |
|---|---|---|---|
| | Discrete | Value-based Reinforcement Learning | Policy-based Reinforcement Learning |
| | Continuous | | |

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | Dynamic Optimization | **Dynamic Game** |

**Action space**

**Time space**

| **Model based** | Finite | Infinite |
|---|---|---|
| Discrete | Markov Game (Stochastic Game) | DT Infinite dynamic game (Stochastic Game) |
| Continuous | Continuous time Markov Game | CT-time Infinite dynamic game (differential game) |

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | Dynamic Optimization | **Dynamic Game** |

**Action space**

**Time space**

| **Model free** | Finite | Infinite |
|---|---|---|
| Discrete | Multi-Agent Value-based RL | Multi-Agent Policy-based RL |
| Continuous | | |

## Discrete-Time Optimal Control Problems

Want to find **a control sequence** $u = \{u_k, k \in \mathbf{K}\}$ which minimizes

$$L(u) = \sum_{k=0}^{K-1} g_k(x_k, u_k) + g_K(x_K) \qquad (1)$$

where the state variable $x_k$ satisfies discrete-time system constraints:

$$x_{k+1} = f_k(x_k, u_k), \qquad u_k \in U_k, \qquad k \in \mathbf{K} \qquad (2)$$

on time steps $k \in \mathbf{K} = \{0,1, \dots, K-1\}$

- $u = \{u_k, k \in \mathbf{K}\}, u_k = \gamma_k(x_k)$
  - ✓ $\gamma_k(\cdot)$ is a permissible control strategy at stage $k \in \mathbf{K}$

- In order to determine the minimizing control strategy, we define the expression for the minimum cost from any staring point $x$ at any initial time $k$, which is so called value function

$$V(k,x) = \min_{\gamma_k,\dots,\gamma_K} \left[ \sum_{i=k}^{K} g_i(x_{i+1}, u_i, x_i) \right] \tag{3}$$

with $u_i = \gamma_i(x_i) \in U_i$ and $x_k = x$

- A direct application of the **principle of optimality** now readily leads to the recursive relation

$$V(k,x) = \min_{u_k} \left[ g_k(\overset{x_{k+1}}{f_k(x,u_k)}, u_k, x) + V\left(k+1, \overset{x_{k+1}}{f_k(x,u_k)}\right) \right] \tag{4}$$

- If the optimal control problem admits a solution $u^* = \{\gamma_k^*, k \in \mathbf{K}\}$, then the solution $V(1,x)$ of Eq.(4) should be equal to $L(u^*)$

- Each $u_k^*$ should be determined as an argument of the RHS of Eq.(4)**(single-shot optimization)**

An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

## Infinite horizon linear-quadratic problem (Discrete Time Optimal Control)

- discrete-time system $x_{t+1} = Ax_t + Bu_t,\ x_0 = x^{init}$
- problem: choose $u_0, u_1, \ldots$ to minimize

$$J = \sum_{\tau=0}^{\infty} (x_\tau^T Q x_\tau + u_\tau^T R u_\tau)$$

with given constant state and input weight matrices

$$Q = Q^T \geq 0, \qquad R = R^T > 0$$

✓ this is an infinite dimensional problem

## Infinite horizon linear-quadratic problem (Discrete Time Optimal Control)

- **Problem**: it's possible that $J = \infty$ for all input sequences $u_0, \ldots$

$$x_{t+1} = 2x_t + 0u_t, \qquad x^{init} = 1$$

- Let's assume $(A, B)$ is controllable
- then for any $x^{init}$ there's an input sequence

$$u_0, \ldots, u_{n-1}, 0, 0, \ldots$$

that steers $x$ to zero at $t = n$, and keeps it there

- ✓ For this $u$, $J < \infty$
- ✓ And therefore, $\min_u J < \infty$ for any $x^{init}$

## Infinite horizon linear-quadratic problem (Discrete Time Optimal Control)

- To apply dynamic programming approach define value function $V : \mathbf{R}^n \to \mathbf{R}$

$$V(z) = \min_{u_0,\ldots} \sum_{\tau=0}^{\infty} (x_\tau^T Q x_\tau + u_\tau^T R u_\tau)$$

subject to $x_0 = z, \quad x_{\tau+1} = Ax_\tau + Bu_\tau$

- $V(z)$ is the minimum LQR cost-to-go, starting from state $z$
- doesn't depend on time-to-to, which is always $\infty$; infinite horizon problem is *shift invariant*

## Infinite horizon linear-quadratic problem (Discrete Time Optimal Control)

- **Fact**: $V$ is quadratic, *i.e.*, $V(z) = z^T P z$, where $P = P^T \geq 0$
- Applying Bellman minimum principle (or HJB equation for discrete system)

$$V(z) = \min_w \left( z^T Q z + w^T R w + V(Az + Bw) \right)$$

or

$$z^T P z = \min_w \left( z^T Q z + w^T R w + (Az + Bw)^T P (Az + Bw) \right)$$

- minimizing $w^* = -(R + B^T P B)^{-1} B^T P A z$
- so HJB equation is

$$z^T P z = z^T Q z + w^{*T} R w^* + (Az + Bw^*)^T P (Az + Bw^*)$$

$$= z^T (Q + A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A) z$$

✓ This must hold for all $z$, so we can conclude that $P$ satisfies the **ARE**

$$P = Q + A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A$$

- The optimal input is constant state feedback $u_t = K x_t$

$$K = -(R + B^T P B)^{-1} B^T P A$$

ARE: Algebraic Riccati Equation

## Infinite horizon linear-quadratic problem (Discrete Time Optimal Control)

- **Fact**: the ARE has only one positive semidefinite solution $P$
  - ✓ ARE plus $P = P^T \geq 0$ uniquely characterizes value function
- Consequence: The Riccati recursion

$$P_{k+1} = Q + A^T P_k A - A^T P_k B(R + B^T P_k B)^{-1} B^T P_k A, \qquad P_1 = Q$$

converges to the unique PSD solution of the ARE (when (A,B) controllable)

- Infinite-horizon LQR optimal control is same as steady-state finite horizon optimal control

## Continuous-Time Optimal Control Problems

Consider the following optimal control problem defined by

$$L(u) = \int_0^T g\big(t, x(t), u(t)\big)\mathrm{d}t + q(T, x(T)) \tag{5}$$

where the state variable $x(t)$ satisfies the differential equation:

$$\dot{x}(t) = f\big(t, x(t), u(t)\big), \qquad x(0) = x_0, \qquad t \geq 0 \tag{6}$$

- $u(t) = \gamma\big(t, x(t)\big) \in U$,
  - ✓ $\gamma \in \Gamma$ is the class of all admissible feedback strategies

- The minimum cost-to-go from any initial state $x$ and any initial time $t$ is described by the so-called ***value function*** defined by

$$V(t,x) = \min_{u(s),\, t \le s \le T} \left[ \int_t^T g\big(s, x(s), u(s)\big)\mathrm{d}s + q\big(T, x(T)\big) \right] \tag{7}$$

Satisfying the boundary condition

$$V(T,x) = q\big(T, x(T)\big) \tag{8}$$

## Hamilton-Jacobi-Bellman (HJB) equation

- The dynamic programming approach, when applied to optimal control problems defined in continuous time, leads to a partial differential equation (PDE) which is know as the ***Hamilton-Jacobi-Bellman (HJB) equation***.

- A direct application of ***the principle of optimality*** on Eq. (7), under the assumption of continuous differentiability of $V$, leads to the HJB equation

$$-\frac{\partial V(t,x)}{\partial t} = \min_{u}\left[\frac{\partial V(t,x)}{\partial x} f(t,x,u) + g(t,x,u)\right] \qquad (9)$$

with $V(T,x) = q\big(T, x(T)\big)$ boundary condition

- In general, it is not easy to compute $V(t,x)$ and the continuous differentiability assumption is rather restrictive.

- If $V(t,x)$ exists, the HJB equation provides a means of obtaining the optimal control strategy **(sufficient condition)**

## Proof of Hamilton-Jacobi-Bellman (HJB) equation

**Proof:**

According to Bellman's optimality principle, the following identity holds for small $\delta$

$$V\big(t,x(t)\big)=\min_{u\in U}\big[g(t,x,u)\delta+V\big(t+\delta,x(t+\delta)\big)\big]$$

where $V(t+\delta,x(t+\delta))=V(t+\delta,x(t)+f(t,x,u)\cdot\delta)$

$$=V(t,x(t))+\frac{\partial V(t,x)}{\partial t}\delta+\frac{\partial V(t,x)}{\partial x}f(t,x,u)\cdot\delta+o(\delta)$$

Substituting into (3) gives

$$V\big(t,x(t)\big)=\min_{u\in U}\left[g(t,x,u)\cdot\delta+V\big(t,x(t)\big)+\frac{\partial V(t,x)}{\partial x}f(t,x,u)\cdot\delta+\frac{\partial V(t,x)}{\partial t}\delta+o(\delta)\right]$$

Canceling $V(t,x(t))$ both side and divide by $\delta$ gives:

$$-\frac{\partial V(t,x)}{\partial t}=\min_{u}\left[\frac{\partial V(t,x)}{\partial x}f(t,x,u)+g(t,x,u)\right]$$

## Obtaining the optimal control strategy from HJB equation

**Theorem**

If a continuously differentiable function $V(t,x)$ can be found that satisfies the HJB equation (9), then it generate the optimal strategy through the static (pointwise) minimization problem defined by the RHS of (9)

$$-\frac{\partial V(t,x)}{\partial t} = \min_{u \in U}\left[\frac{\partial V(t,x)}{\partial x} f(t,x,u) + g(t,x,u)\right] \quad (9)$$

$$u^*(t) = \operatorname*{argmin}_{u \in U}\left[\frac{\partial V(t,x)}{\partial x} f(t,x,u) + g(t,x,u)\right] \quad (10)$$

## Obtaining the optimal control strategy from HJB equation

**Proof:**

- If we are given two strategies, $\gamma^* \in \Gamma$ (the optimal one) and $\gamma \in \Gamma$ (an arbitrary one), with the corresponding terminating trajectories $x^*$ and $x$, and terminal times $T^*$ and $T$, respectively, then Eq. (9) reads

$$g(t,x,u) + \frac{\partial V(t,x)}{\partial x} f(t,x,u) + \frac{\partial V(t,x)}{\partial t} \geq 0 \qquad (11)$$

$$g(t,x^*,u^*) + \frac{\partial V(t,x^*)}{\partial x} f(t,x^*,u^*) + \frac{\partial V(t,x^*)}{\partial t} \equiv 0 \qquad (12)$$

  where $\gamma^*$ and $\gamma$ have been replaced by the corresponding controls $u^*$ and $u$, respectively.

- Integrating Eq.(11) from 0 to $T$ and Eq.(12) from 0 to $T^*$ leads to

$$\int_0^T g(t,x,u) + V(T,x(T)) - V(0,x_0) \geq 0$$

$$\int_0^{T^*} g(t,x^*,u^*) + V(T,x^*(T^*)) - V(0,x_0) = 0$$

- Elimination of $V(0,x_0)$ yields

$$\int_0^T g(t,x,u) + q(T,x(T)) \geq \int_0^{T^*} g(t,x^*,u^*) + q(T^*,x^*(T^*)) \geq 0$$

➢ $u^*$ is the optimal control (sequence of actions), and therefore $\gamma^*$ is the optimal strategy

## Infinite horizon linear-quadratic problem (Continuous Time Optimal Control)

- continuous-time system $\dot{x}(t) = Ax(t) + Bu(t)$
- problem: choose $u(t)$ to minimize

$$J(u) = \int_0^T \left(x(t)^T Q x(\mathrm{t}) + u(\mathrm{t})^T R u(\mathrm{t})\right) d\mathrm{t} + x(T)^T Q_f x(T)$$

  - ✓ $g = x^T Q x + u^T R u$
  - ✓ $f = Ax + Bu$
  - ✓ Let's assume $V(t,x) = x^T P_t x$

- HJB equation is employed

$$-\frac{\partial V(t,x)}{\partial t} = \min_{u \in U}\left[\frac{\partial V(t,x)}{\partial x} f(t,x,u) + g(t,x,u)\right]$$

$$\Rightarrow -x\dot{P}_t x = \min_u \{2P_t x(Ax + Bu) + x^T Q x + u^T R u\}$$

- minimizing over $u$ yields $u^* = -R^{-1}B^T P_t$, and substitute to HJB;

$$-\dot{P}_t = A^T P_t + P_t A - P_t B R^{-1} B^T P_t + Q$$

which is called as ***Riccati differential equation*** in optimal control.

# 2. Policy-Based Reinforcement Learning

# Overview

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | **Dynamic Optimization** | Dynamic Game |

**Action space**

| **Time space** | **Model based** | Finite | Infinite |
|---|---|---|---|
| | Discrete | Discrete time MDP $P(s_{t+1}\|s_t, a_t)$ | Discrete-time dynamic system $x_{t+1} = f(x_t, u_t)$ |
| | Continuous | Continuous time MDP $P(s_{t+h}\|s_t, a_t)$ | Continuous-time dynamic system $\dot{x}_t = f(x_t, u_t)$ |

# Overview

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | **Dynamic Optimization** | Dynamic Game |

**Action space**

| **Time space** | **Model free** | Finite | Infinite |
|---|---|---|---|
| | Discrete | Value-based Reinforcement Learning | Policy-based Reinforcement Learning |
| | Continuous | | |

# Overview

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | Dynamic Optimization | **Dynamic Game** |

**Action space**

**Time space**

| **Model based** | Finite | Infinite |
|---|---|---|
| Discrete | Markov Game (Stochastic Game) | DT Infinite dynamic game (Stochastic Game) |
| Continuous | Continuous time Markov Game | CT-time Infinite dynamic game (differential game) |

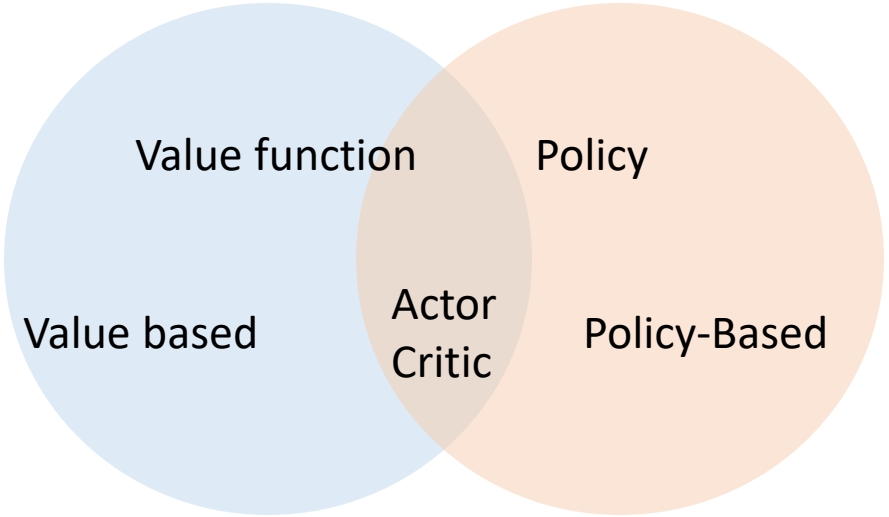
# Overview

| | Single Agent | Multi Agent |
|---|---|---|
| Static | Static optimization | Static Game |
| Dynamic | Dynamic Optimization | **Dynamic Game** |

**Action space**

| **Time space** | **Model free** | Finite | Infinite |
|---|---|---|---|
| | Discrete | Multi-Agent Value-based RL | Multi-Agent Policy-based RL |
| | Continuous | | |

# Overview


Value function
Policy
Value based
Actor
Critic
Policy-Based

# Motivation

- One of the primary goals of the field of artificial intelligence is to solve complex tasks from unprocessed, high-dimensional, sensory input.

- Recent progresses in RL have shown the possibilities
  - ✓ Deep Q Network for Atari games and
  - ✓ AlphaGo

- Although DQN solves problems with high-dimensional observation spaces, it can only handle discrete and low-dimensional action spaces

# Policy Gradient

- Policy Gradient methods learn the policy directly with a parameterized function respect to $\theta$, $\pi_\theta(a|s)$.
- The reward function can be defined as (in continuous case):

$$J(\theta) = \sum_{s \in \mathcal{S}} d^\pi(s) V^\pi(s) = \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a)$$

  - ✓ where $d^\pi(s)$ is stationary distribution of Markov chain for $\pi_\theta$ (on-policy state distribution under $\pi$)

$$d^\pi(s) = \lim_{t \to \infty} P(s_t = s | s_0, \pi_\theta)$$

  - ✓ $d^\pi = d^{\pi_\theta}$; $V^\pi = V^{\pi_\theta}$; $Q^\pi = Q^{\pi_\theta}$ for simplicity

- We can find the best $\theta^*$ using *gradient ascent* such that

$$\theta^* = \underset{\theta}{\operatorname{argmax}} J(\theta)$$

- It is natural to expect policy-based methods are more useful in the continuous space
  - ✓ There is an infinite number of actions and (or) states to estimate the values
  - ✓ Difficult to compute $\underset{a \in \mathcal{A}}{\operatorname{argmax}} Q^\pi(s,a)$ requiring full scan of the action space

# How to compute the gradient $\nabla J(\theta)$?

- Gradient can be computed by perturbing $\theta$ by a small amount $\epsilon$ in the $k$-th dimension.

$$\frac{\partial J(\theta)}{\partial \theta_k} \approx \frac{J(\theta + \epsilon u_k) - J(\theta)}{\epsilon}$$

  - ✓ It works even when $J(\theta)$ is not differentiable, but very slow

- Is there more efficient method?

# Policy Gradient Theorem

- The reward function is

$$J(\theta) = \sum_{s \in \mathcal{S}} d^\pi(s) V^\pi(s) = \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a)$$

- Computing the gradient $\nabla J(\theta)$ is tricky because it depends on both the action selection and the stationary distribution of states following the target selection behavior
- The **policy gradient theorem** makes the computation of $\nabla J(\theta)$ simple

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a) \\
&\propto \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) \qquad \text{gradient does not depends on } Q^\pi \text{ (will be proved)} \\
&= \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} Q^\pi(s,a) \\
&= \sum_{s} d^\pi(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) \nabla \ln \pi_\theta(a|s) Q^\pi(s,a) \qquad \because (\ln x)' = 1/x \\
&= \mathbb{E}_\pi[\nabla \ln \pi_\theta(a|s) Q^\pi(s,a)]
\end{aligned}$$

✓ where $\mathbb{E}_\pi$ refers to $\mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta}$ when both state and action distributions follow the policy $\pi_\theta$ (**on-policy**)

# Policy Gradient Theorem Proof

$$\nabla_\theta V^\pi(s) = \nabla_\theta \left( \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a) \right)$$

$$= \sum_{a \in \mathcal{A}} \left( \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) + \pi_\theta(a|s) \nabla_\theta Q^\pi(s,a) \right) \quad \because \text{derivative product rule}$$

$$= \sum_{a \in \mathcal{A}} \left( \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) + \pi_\theta(a|s) \nabla_\theta \sum_{s',r} P(s',r|s,a)(r + V^\pi(s')) \right) \quad \because \text{Extend } Q^\pi \text{ with further state value}$$

$$= \sum_{a \in \mathcal{A}} \left( \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) + \pi_\theta(a|s) \sum_{s',r} P(s',r|s,a) \nabla_\theta V^\pi(s') \right) \quad \because P(s',r|s,a) \text{ is not a function of } \theta$$

$$= \sum_{a \in \mathcal{A}} \left( \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) + \pi_\theta(a|s) \sum_{s'} P(s'|s,a) \nabla_\theta V^\pi(s') \right) \quad \because P(s'|s,a) = \sum_r P(s',r|s,a)$$

$$\nabla_\theta V^\pi(s) = \sum_{a \in \mathcal{A}} \left( \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) + \pi_\theta(a|s) \sum_{s'} P(s'|s,a) \nabla_\theta V^\pi(s') \right)$$

✓ a nice recursive form and the future state value function $V^\pi(s')$ can be repeated unrolled as will be shown next slide

# Policy Gradient Theorem Proof

$$\nabla_\theta V^\pi(s) = \sum_{a\in\mathcal{A}} \left( \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) + \pi_\theta(a|s) \sum_{s'} P(s'|s,a) \nabla_\theta V^\pi(s') \right)$$

$$= \phi(s) + \sum_{a\in\mathcal{A}} \pi_\theta(a|s) \sum_{s'} P(s'|s,a) \nabla_\theta V^\pi(s') \qquad \phi(s) = \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a)$$

$$= \phi(s) + \sum_{s'} \sum_{a} \pi_\theta(a|s) P(s'|s,a) \nabla_\theta V^\pi(s')$$

$$= \phi(s) + \sum_{s'} \rho^\pi(s \to s', 1)\, \nabla_\theta V^\pi(s')$$

$$= \phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) \left[ \phi(s') + \sum_{s''} \rho^\pi(s' \to s'', 1)\, \nabla_\theta V^\pi(s'') \right]$$

$$= \phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) \phi(s') + \sum_{s''} \rho^\pi(s \to s'', 2)\, \nabla_\theta V^\pi(s'')$$

$$= \phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) \phi(s') + \sum_{s''} \rho^\pi(s \to s'', 2)\, \phi(s'') + \sum_{s'''} \rho^\pi(s \to s''', 3)\, \nabla_\theta V^\pi(s''')$$

$= \cdots$; repeatedly unrolling the part of $\nabla_\theta V^\pi(\cdot)$

$$= \sum_{x\in\mathcal{S}} \sum_{k}^{\infty} \rho^\pi(s \to x, k) \phi(x)$$

# Policy Gradient Theorem Proof

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta V^\pi(s_0) \\
&= \sum_s \sum_k^\infty \rho^\pi(s_0 \to s, k)\phi(s) \\
&= \sum_s \eta(s)\phi(s) && \because\ \eta(s) = \sum_{k=0}^\infty \rho^\pi(s_0 \to s, k) \\
&= \left(\sum_s \eta(s)\right) \sum_s \frac{\eta(s)}{(\sum_s \eta(s))}\phi(s) && \because \text{Normalize } \eta(s), s \in \mathcal{S} \text{ to be a probability dist.} \\
&\propto \sum_s \frac{\eta(s)}{(\sum_s \eta(s))}\phi(s) && \because\ \textstyle\sum_s \eta(s) \text{ is a constant} \\
&= \sum_s d^\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) && \because\ \phi(s) = \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) \\
&= \sum_s d^\pi(s) \sum_{a\in\mathcal{A}} \pi_\theta(a|s) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} Q^\pi(s,a) \\
&= \sum_s d^\pi(s) \sum_{a\in\mathcal{A}} \pi_\theta(a|s) \nabla_\theta \ln \pi_\theta(a|s) Q^\pi(s,a) \\
&= \mathbb{E}_\pi[\nabla_\theta \ln \pi_\theta(a|s) Q^\pi(s,a)]
\end{aligned}
$$

$\mathbb{E}_\pi$ refers to $\mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}$ (**on-policy**)

# Generalized Advantage Estimation (GAE) (Schulman et al., 2016)

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi[\nabla_\theta \ln \pi_\theta(a|s) Q^\pi(s,a)]$$

- The policy gradient theorem lays the theoretical foundation for various policy gradient algorithms.
- This vanilla policy gradient update has no bias but high variance. Many following algorithms were proposed to reduce the variance while keeping the bias unchanged.
  - ✓ Reinforce
  - ✓ Actor-Critic
  - ✓ Off-Policy Policy Gradient
  - ✓ High-dimensional Continuous Control Using Generalized Advantage Estimation (Schulman, et al., 2016)
  - ✓ Asynchronous Advantage Actor Critic (Miih et al., 2016)
  - ✓ Deterministic Policy Gradient (DPG)
  - ✓ Deep Deterministic Policy Gradient (DDPG) (Lillicrap, et al., 2015)
  - ✓ Addressing Function Approximation Error in Actor-Critic Methods (TD3) (Fujimoto, et al., 2018)
  - ✓ Trust region policy optimization (TRPO) (Schulman, et al., 2015)
  - ✓ Proximal Policy Optimization (PPO) (Schulman, et al., 2017)
  - ✓ Soft Actor Critic (SAC) (Haarnoja et al., 2018)
  - ✓ Multi-Agent Deep Deterministic Policy Gradient (MADDPG) (Lowe et al., 2017)

# REINFORCE Algorithm

- REINFORCE (Monte-Carlo policy gradient) relies on an estimated return by Monte-Carlo methods using episode samples to update the policy parameter $\theta$
- REINFORCE works because the expectation of the sample gradient is equal to the actual gradient:

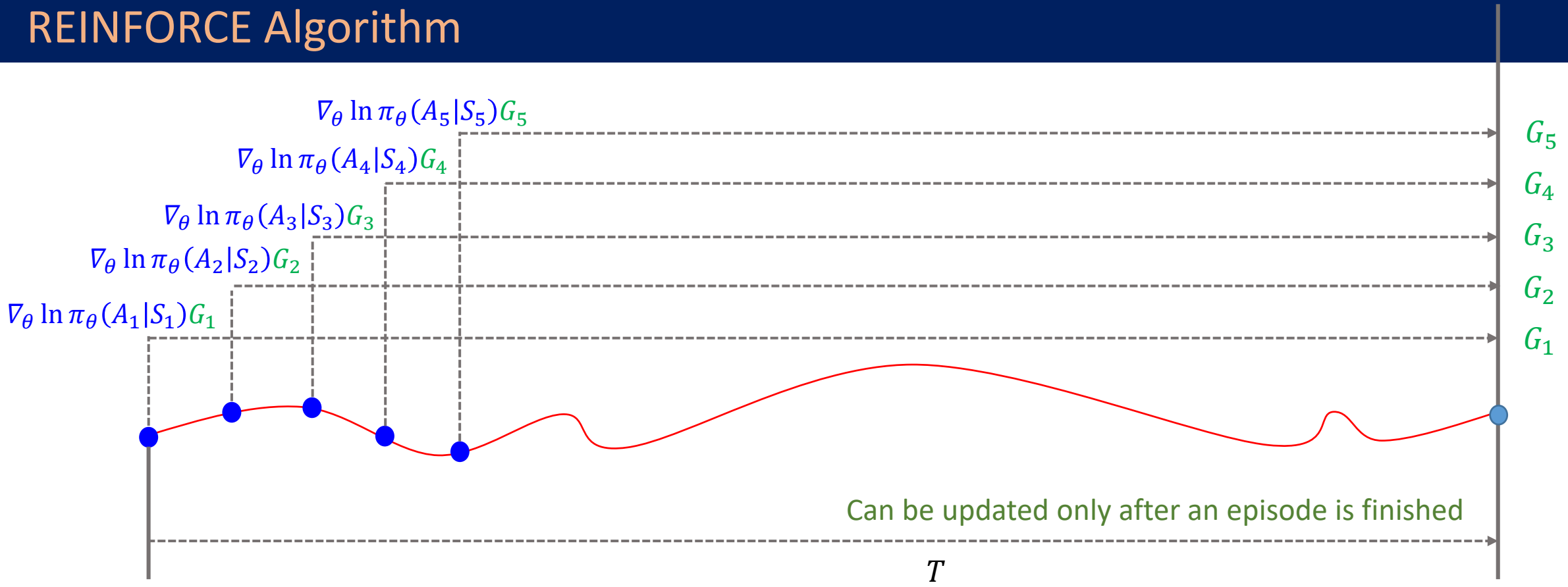
$$\nabla_\theta J(\theta) = \mathbb{E}_\pi[\nabla_\theta \ln \pi_\theta(a|s) Q^\pi(s,a)]$$
$$= \mathbb{E}_\pi[\nabla_\theta \ln \pi_\theta(A_t|S_t) G_t] \qquad \because Q^\pi(s,a) = \mathbb{E}_\pi[G_t|S_t, A_t]$$

  - ✓ Measure $G_t$ from real sample trajectories and use that to update policy gradient
  - ✓ It relies on a full trajectory and that's why it is a Monte-Carlo method

1. Initialize the policy parameter θ at random.
2. Generate one trajectory on policy $\pi_\theta$: $S_1, A_1, R_2, S_2, A_2, \ldots, S_T$.
3. For t=1, 2, … , T:
    1. Estimate the the return $G_t$;
    2. Update policy parameters: $\theta \leftarrow \theta + \alpha\gamma^t G_t \nabla_\theta \ln \pi_\theta(A_t|S_t)$

- A widely used variation of REINFORCE is to **subtract a baseline value** from the return $G_t$ to reduce the variance of gradient estimation while keeping the bias unchanged

# REINFORCE Algorithm




- Update per every action

$$\theta \leftarrow \theta + \alpha\gamma^t \nabla_\theta \ln \pi_\theta(A_t|S_t)G_t$$

- Update per every episode

$$\theta \leftarrow \theta + \alpha\left[\sum_{t=1}^{T} \gamma^t \nabla_\theta \ln \pi_\theta(A_t|S_t)G_t\right]$$

- Update per episode & batch

$$\theta \leftarrow \theta + \alpha\left[\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T} \gamma^t \nabla_\theta \ln \pi_\theta\left(A_t^{(i)}\middle|S_t^{(i)}\right)G_t^{(i)}\right]$$

# Actor-Critic

- It makes a lot of sense to learn the value function in addition to the policy, since knowing the value function can assist the policy update, such as by reducing gradient variance in vanilla policy gradients
- Two main components in policy gradient are the policy model and the value function.
  - Critic updates the value function parameters $w$ and depending on the algorithm it could be action value $Q_w(s,a)$ or state-value $V_w(s)$
  - Actor updates the policy parameters $\theta$ for $\pi_\theta(a|s)$, in the direction suggested by the critic

1. Initialize s, θ, w at random; sample $a \sim \pi_\theta(a|s)$.
2. For $t = 1 \ldots T$:
   1. Sample reward $r_t \sim R(s,a)$ and next state $s' \sim P(s'|s,a)$;
   2. Then sample the next action $a' \sim \pi_\theta(a'|s')$;
   3. Update the policy parameters: $\theta \leftarrow \theta + \alpha_\theta Q_w(s,a) \nabla_\theta \ln \pi_\theta(a|s)$;
   4. Compute the correction (TD error) for action-value at time t:
      $\delta_t = r_t + \gamma Q_w(s',a') - Q_w(s,a)$
      and use it to update the parameters of action-value function:
      $w \leftarrow w + \alpha_w \delta_t \nabla_w Q_w(s,a)$
   5. Update $a \leftarrow a'$ and $s \leftarrow s'$.

(on-policy learning)

# Off-Policy Policy Gradient (Degris, White & Sutton, 2012).

- Both REINFORCE and the vanilla version of actor-critic method are on-policy:
  - ✓ training samples are collected according to the target policy — the very same policy that we try to optimize for.
- Off policy methods, however, result in several additional advantages:
  - ✓ The off-policy approach does not require full trajectories and can reuse any past episodes (experience replay) for much better sample efficiency.
  - ✓ The sample collection follows a ***behavior policy*** different from the target policy, bringing ***better exploration***

# Off-Policy Policy Gradient

$$J(\theta) = \sum_{s \in \mathcal{S}} d^{\beta}(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a) = \mathbb{E}_{s \sim d^\beta} \left[ \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a) \right]$$

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{s \sim d^\beta} \left[ \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a) \right] \\
&= \mathbb{E}_{s \sim d^\beta} \left[ \sum_{a \in \mathcal{A}} \left( \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) + \pi_\theta(a|s) \nabla_\theta Q^\pi(s,a) \right) \right] \\
&\approx \mathbb{E}_{s \sim d^\beta} \left[ \sum_{a \in \mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) \right] \\
&= \mathbb{E}_{s \sim d^\beta} \left[ \sum_{a \in \mathcal{A}} \beta(a|s) \frac{\pi_\theta(a|s)}{\beta(a|s)} \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} Q^\pi(s,a) \right] \\
&= \mathbb{E}_{\beta} \left[ \frac{\pi_\theta(a|s)}{\beta(a|s)} \nabla_\theta \ln \pi_\theta(a|s) \, Q^\pi(s,a) \right]
\end{aligned}$$

Importance weight

Derivative product rule

Ignore $\pi_\theta(a|s) \nabla_\theta Q^\pi(s,a)$

- if we use an approximated gradient with the gradient of Q ignored, we still guarantee the policy improvement and eventually achieve the true local minimum (Degris, White & Sutton, 2012).
- In summary, when applying policy gradient in the off-policy setting, we can simple adjust it with a weighted sum and the weight is the ratio of the target policy to the behavior policy,

Policy gradient methods maximize the expected total reward by repeatedly estimating the gradient $g := \nabla_\theta \mathbb{E}\left[\sum_{t=0}^{\infty} r_t\right]$. There are several different related expressions for the policy gradient, which have the form

$$g = \mathbb{E}\left[\sum_{t=0}^{\infty} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right], \tag{1}$$

where $\Psi_t$ may be one of the following:

1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory.
2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action $a_t$.
3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula.
4. $Q^\pi(s_t, a_t)$: state-action value function.
5. $A^\pi(s_t, a_t)$: advantage function.
6. $r_t + V^\pi(s_{t+1}) - V^\pi(s_t)$: TD residual.

The latter formulas use the definitions

$$V^\pi(s_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t:\infty}}}\left[\sum_{l=0}^{\infty} r_{t+l}\right] \qquad Q^\pi(s_t, a_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t+1:\infty}}}\left[\sum_{l=0}^{\infty} r_{t+l}\right] \tag{2}$$

$$A^\pi(s_t, a_t) := Q^\pi(s_t, a_t) - V^\pi(s_t), \quad \text{(Advantage function)}. \tag{3}$$

# High-dimensional Continuous Control Using Generalized Advantage Estimation (Schulman, et al., 2016)

We will introduce a parameter $\gamma$ that allows us to reduce variance by downweighting rewards corresponding to delayed effects, at the cost of introducing bias. This parameter corresponds to the discount factor used in discounted formulations of MDPs, but we treat it as a variance reduction parameter in an undiscounted problem; this technique was analyzed theoretically by Marbach & Tsitsiklis (2003); Kakade (2001b); Thomas (2014). The discounted value functions are given by:

$$V^{\pi,\gamma}(s_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t:\infty}}}\left[\sum_{l=0}^{\infty}\gamma^l r_{t+l}\right] \qquad Q^{\pi,\gamma}(s_t,a_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t+1:\infty}}}\left[\sum_{l=0}^{\infty}\gamma^l r_{t+l}\right] \tag{4}$$

$$A^{\pi,\gamma}(s_t,a_t) := Q^{\pi,\gamma}(s_t,a_t) - V^{\pi,\gamma}(s_t). \tag{5}$$

The discounted approximation to the policy gradient is defined as follows:

$$g^{\gamma} := \mathbb{E}_{\substack{s_{0:\infty}\\ a_{0:\infty}}}\left[\sum_{t=0}^{\infty} A^{\pi,\gamma}(s_t,a_t)\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]. \tag{6}$$

# High-dimensional Continuous Control Using Generalized Advantage Estimation (Schulman, et al., 2016)

Next, let us consider taking the sum of $k$ of these $\delta$ terms, which we will denote by $\hat{A}_t^{(k)}$

$$\hat{A}_t^{(1)} := \delta_t^V = -V(s_t) + r_t + \gamma V(s_{t+1}) \tag{11}$$

$$\hat{A}_t^{(2)} := \delta_t^V + \gamma\delta_{t+1}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \tag{12}$$

$$\hat{A}_t^{(3)} := \delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3}) \tag{13}$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k}) \tag{14}$$

The generalized advantage estimator $\mathrm{GAE}(\gamma, \lambda)$ is defined as the exponentially-weighted average of these $k$-step estimators:

$$\begin{aligned}
\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} &:= (1-\lambda)\left(\hat{A}_t^{(1)} + \lambda\hat{A}_t^{(2)} + \lambda^2\hat{A}_t^{(3)} + \dots\right) \\
&= (1-\lambda)(\delta_t^V + \lambda(\delta_t^V + \gamma\delta_{t+1}^V) + \lambda^2(\delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V) + \dots) \\
&= (1-\lambda)(\delta_t^V(1 + \lambda + \lambda^2 + \dots) + \gamma\delta_{t+1}^V(\lambda + \lambda^2 + \lambda^3 + \dots) \\
&\qquad + \gamma^2\delta_{t+2}^V(\lambda^2 + \lambda^3 + \lambda^4 + \dots) + \dots) \\
&= (1-\lambda)\left(\delta_t^V\left(\frac{1}{1-\lambda}\right) + \gamma\delta_{t+1}^V\left(\frac{\lambda}{1-\lambda}\right) + \gamma^2\delta_{t+2}^V\left(\frac{\lambda^2}{1-\lambda}\right) + \dots\right) \\
&= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V
\end{aligned} \tag{16}$$

Using the generalized advantage estimator, we can construct a biased estimator of $g^\gamma$, the discounted policy gradient from Equation (6):

$$g^\gamma \approx \mathbb{E}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}\right] = \mathbb{E}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V\right], \tag{19}$$

# Asynchronous Advantage Actor Critic (Miih et al., 2016)

- Deep RL algorithms based on experience replay have achieved unprecedented success in challenging domains such as Atari 2600. However, experience replay has several drawbacks:
  - It uses **more memory** and commutation per real interaction
  - It requires off-policy learning algorithms that can update from data generated by an **older policy**
- Instead of experience replay, A3C asynchronously executes **multiple gents in parallel**, on multiple instances of the environment
  - **Parallelism decorrelates the agents' data input** a more stationary processes, since at any given time step the parallel agents will be experiencing a variety of different states.
  - Parallelism stabilizes learning, playing a role of experience replay buffer
- Make the observation that multiple actor-learners running in parallel are likely to be exploring different parts of the environment
  - One can explicitly use different exploration policies in each actor-learners to maximize the diversity
  - Make samples less correlated

# Asynchronous Advantage Actor Critic (Miih et al., 2016)

**Algorithm S3** Asynchronous advantage actor-critic - pseudocode for each actor-learner thread.

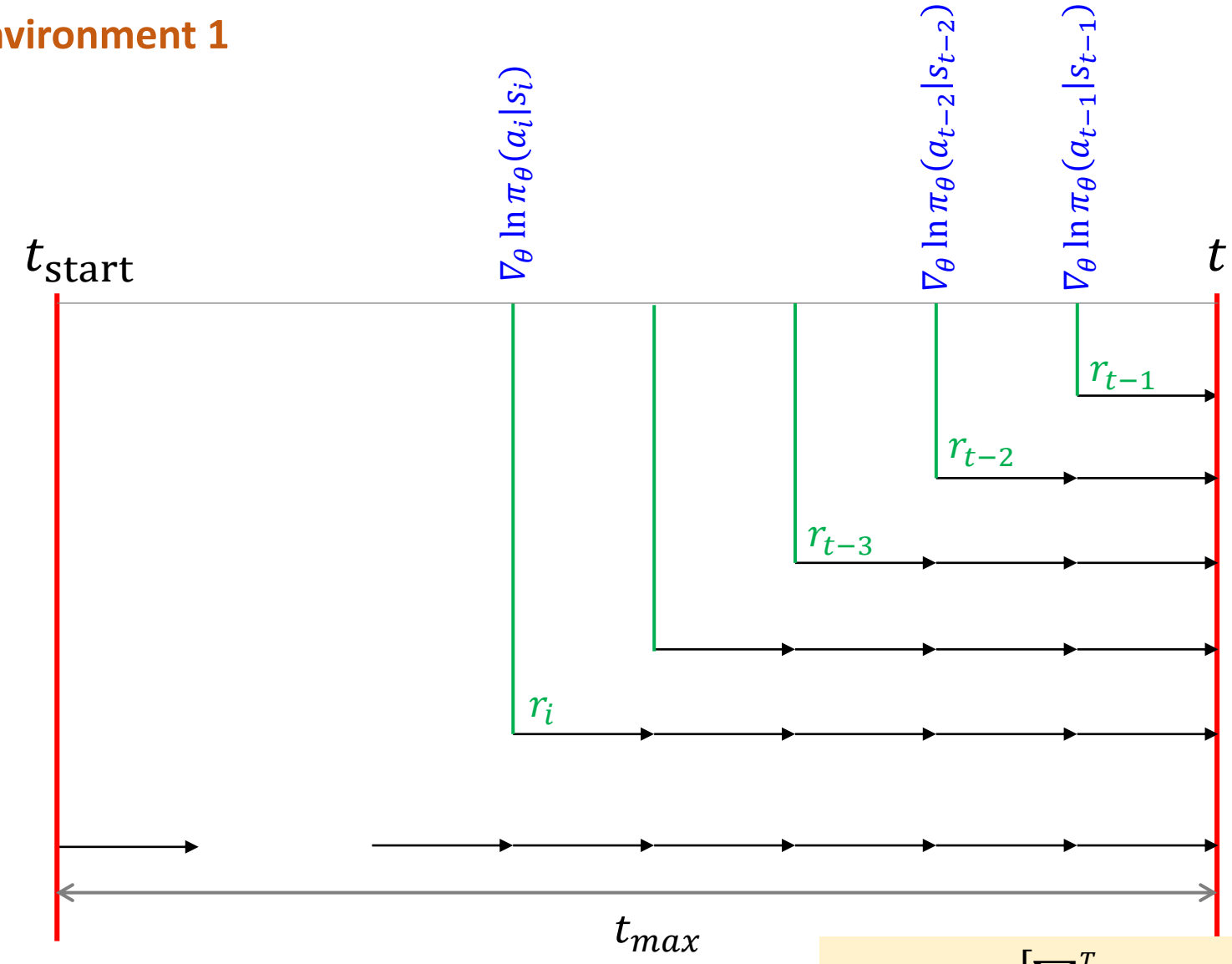
*// Assume global shared parameter vectors* $\theta$ *and* $\theta_v$ *and global shared counter* $T = 0$
*// Assume thread-specific parameter vectors* $\theta'$ *and* $\theta'_v$
Initialize thread step counter $t \leftarrow 1$
**repeat**
  Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
  Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$
  $t_{start} = t$
  Get state $s_t$
  **repeat**
    Perform $a_t$ according to policy $\pi(a_t|s_t;\theta')$
    Receive reward $r_t$ and new state $s_{t+1}$
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  **until** terminal $s_t$ **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t \text{// Bootstrap from last state} \end{cases}$
  **for** $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt $\theta'$: $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i;\theta')(R - V(s_i;\theta'_v))$
    Accumulate gradients wrt $\theta'_v$: $d\theta_v \leftarrow d\theta_v + \partial \left(R - V(s_i;\theta'_v)\right)^2 / \partial \theta'_v$
  **end for**
  Perform asynchronous update of $\theta$ using $d\theta$ and of $\theta_v$ using $d\theta_v$.
**until** $T > T_{max}$

$$R = \sum_{i=1}^{t_{max}} \gamma^{i-1} r_{t-i} + \gamma^{t_{max}} V(s_t, \theta'_v)$$

- The gradient accumulation step can be considered as a parallelized reformation of mini-batch-based stochastic gradient update

# Asynchronous Advantage Actor Critic (Mnih et al., 2016)

**Environment 1**




$$R = V(s_t, \theta_{v'})$$

$$R \leftarrow r_{t-1} + \gamma V(s_t, \theta_{v'})$$

$$R \leftarrow r_{t-2} + \gamma r_{t-1} + \gamma^2 V(s_t, \theta_{v'})$$

$$R \leftarrow r_{t-3} + \gamma r_{t-2} + \gamma^2 r_{t-1} + \gamma^3 V(s_t, \theta_{v'})$$

$$R = \sum_{i=1}^{t_{max}} \gamma^{i-1} r_{t-i} + \gamma^{t_{max}} V(s_t, \theta'_v)$$

$$\theta \leftarrow \theta + \alpha \left[ \sum_{t=1}^{T} \gamma^t \nabla_\theta \ln \pi_\theta(A_t|S_t)(R_t - V(s_t)) \right]$$

# Asynchronous Advantage Actor Critic (Mnih et al., 2016)

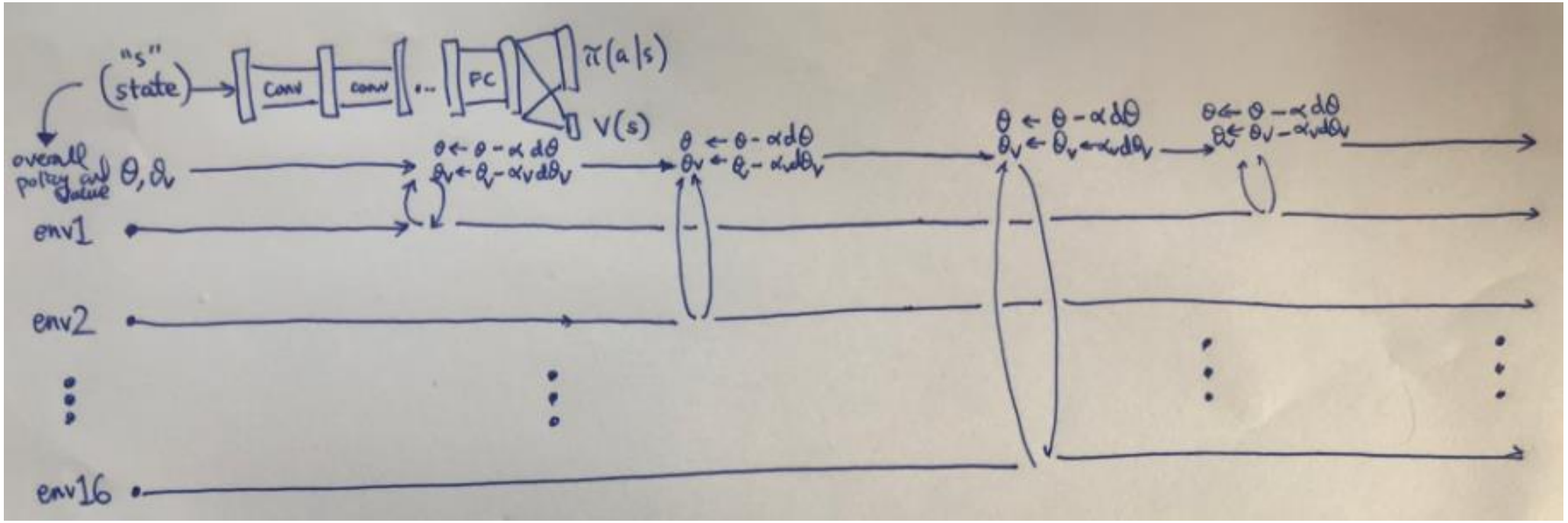
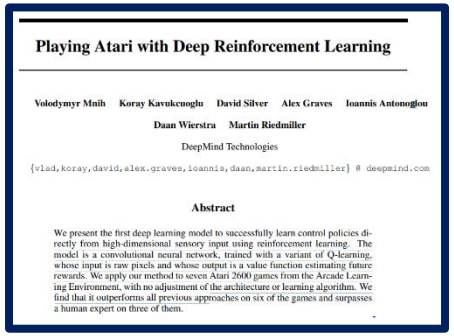



Figure from : https://danieltakeshi.github.io/2018/06/28/a2c-a3c/

# Deep Deterministic Policy Gradient (DDPG) (Lillicrap, et al., 2015)

**DQN**



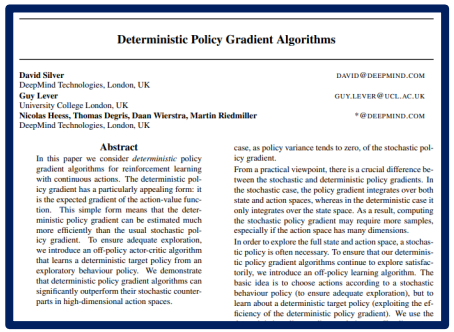
Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou
Daan Wierstra Martin Riedmiller
DeepMind Technologies
{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

**Deterministic Policy Gradient**



Deterministic Policy Gradient Algorithms

David Silver DAVID@DEEPMIND.COM
DeepMind Technologies, London, UK
Guy Lever GUY.LEVER@UCL.AC.UK
University College London, UK
Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller *@DEEPMIND.COM
DeepMind Technologies, London, UK

Abstract

In this paper we consider *deterministic* policy gradient algorithms for reinforcement learning with continuous actions. The deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function. This simple form means that the deterministic policy gradient can be estimated much more efficiently than the usual stochastic policy gradient. To ensure adequate exploration, we introduce an off-policy actor-critic algorithm that learns a deterministic target policy from an exploratory behaviour policy. We demonstrate that deterministic policy gradient algorithms can significantly outperform their stochastic counterparts in high-dimensional action spaces.

case, as policy variance tends to zero, of the stochastic policy gradient.
From a practical viewpoint, there is a crucial difference between the stochastic and deterministic policy gradients. In the stochastic case, the policy gradient integrates over both state and action spaces, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.
In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that our deterministic policy gradient algorithms continue to explore satisfactorily, we introduce an off-policy learning algorithm. The basic idea is to choose actions according to a stochastic behaviour policy (to ensure adequate exploration), but to learn about a deterministic target policy (exploiting the efficiency of the deterministic policy gradient). We use the

**DDPG**

Published as a conference paper at ICLR 2016

CONTINUOUS CONTROL WITH DEEP REINFORCEMENT LEARNING

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess,
Tom Erez, Yuval Tassa, David Silver & Daan Wierstra
Google Deepmind
London, UK
{countzero, jjhunt, apritzel, heess,
etom, tassa, davidsilver, wierstra} @ google.com

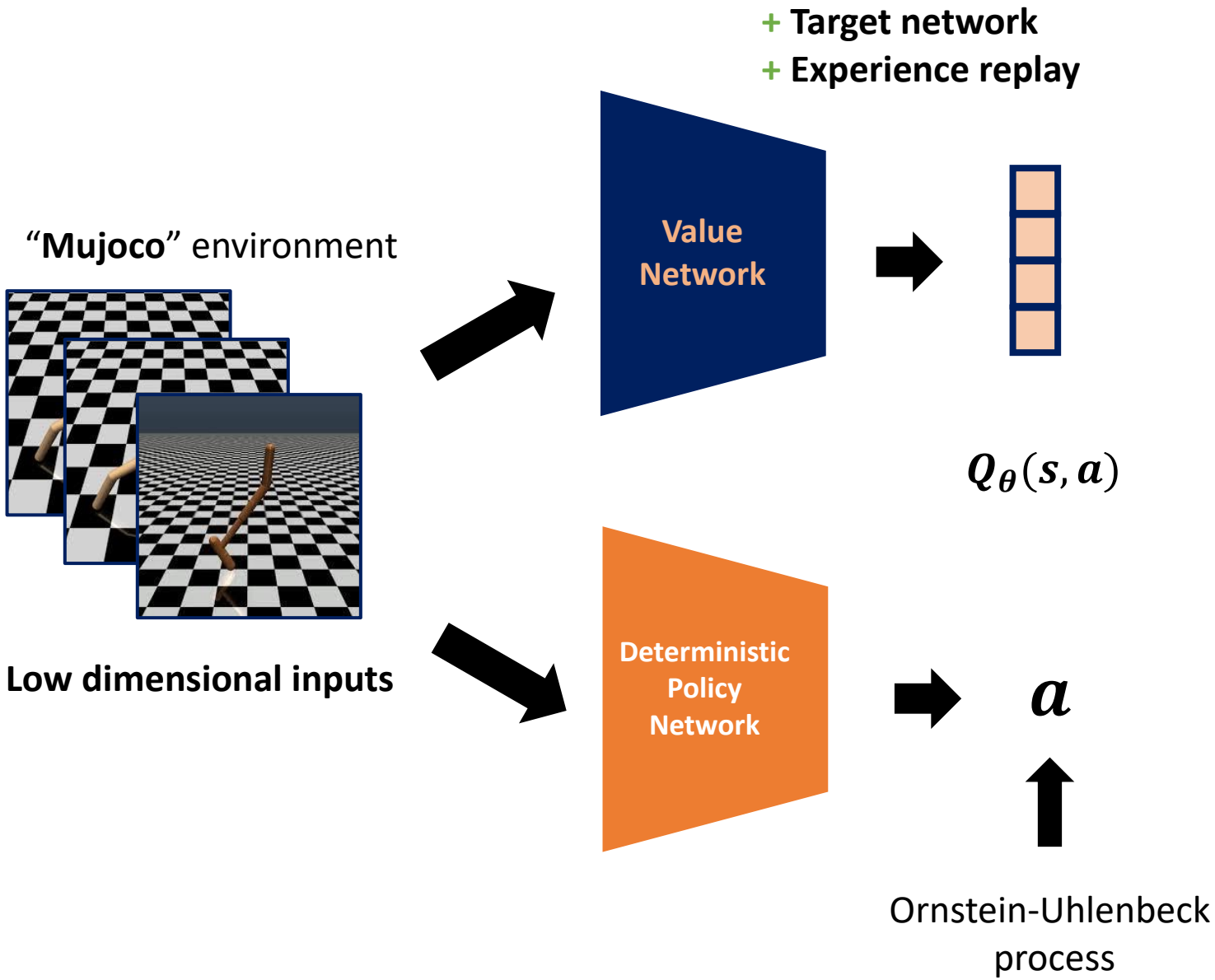
ABSTRACT

We adapt the ideas underlying the success of Deep Q-Learning to the continuous action domain. We present an actor-critic, model-free algorithm based on the deterministic policy gradient that can operate over continuous action spaces. Using the same learning algorithm, network architecture and hyper-parameters, our algorithm robustly solves more than 20 simulated physics tasks, including classic problems such as cartpole swing-up, dexterous manipulation, legged locomotion and car driving. Our algorithm is able to find policies whose performance is competitive with those found by a planning algorithm with full access to the dynamics of the domain and its derivatives. We further demonstrate that for many of the tasks the algorithm can learn policies "end-to-end": directly from raw pixel inputs.

Dec 19, 2013 — ICML 2014 — ICLR 2016

- DDPG is a deep version of "**Deterministic Policy Gradient (2014)**"
- The tricks for stabilizing DQN were also used in DDPG

# DDPG Overview



# DDPG

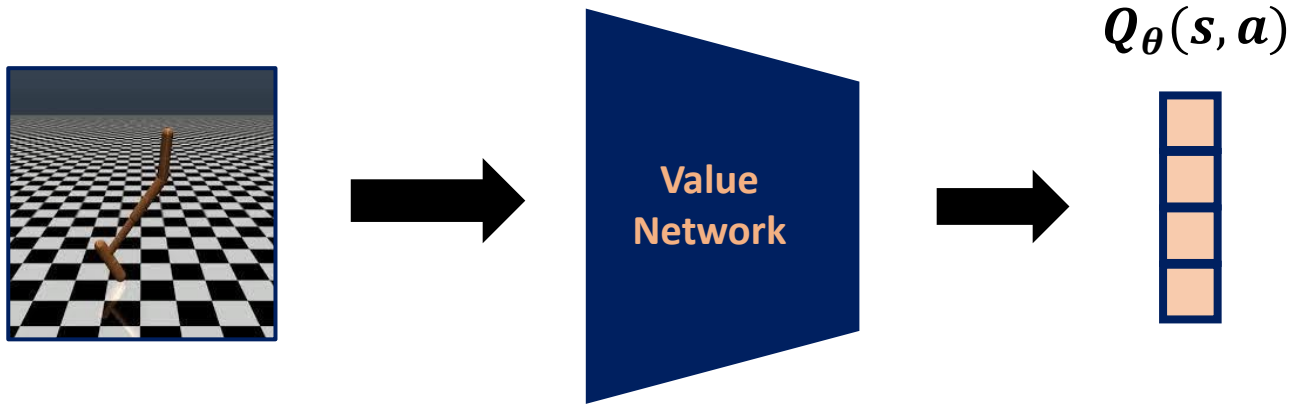
- DDPG combines “DQN” and “Deterministic PG”.
- DDPG can handle continuous action space.
- Deterministic policy (but continuous) enables
  - Off-policy training
  - Sample efficient training
- DDPG utilizes temporally correlated exploration method.
  - Ornstein-Uhlenbeck process

# 1. Deterministic Policy Gradient (Silver et al., 2014)

## Q-Learning on continuous action space?

Option 1) Action discretization

Mujoco action = motor control input of each joint.




Assume the range of control input $-1 \leq a \leq 1$. Then we can discretize the action space as we want.

For instance, $[-1, -1 + \Delta a, -1 + 2\Delta a, \ldots, 1]$. On the discretized action space, we can perform Q-Learning.

Then what is good $\Delta a$? Too small $\Delta a \rightarrow$ Less control resolution / Too high $\Delta a \rightarrow$ Computationally intractable.

Even if we can perform DQN with sufficiently high $\Delta a$, still optimal action couldn't achievable.

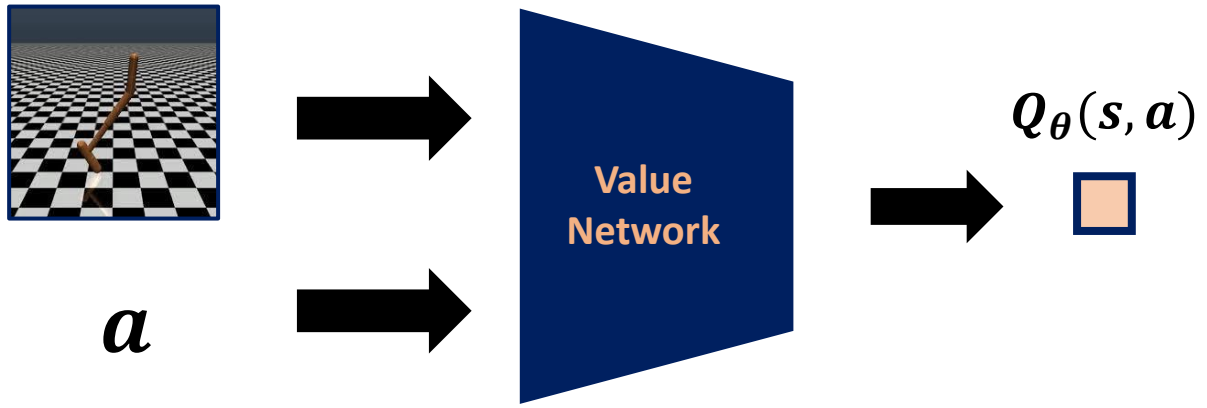
# 1. Deterministic Policy Gradient (Silver et al., 2014)

## Q-Learning on continuous action space?

Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \eta\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$$

Option 1) Action as additional inputs




Assume we decided to use "action" as additional input for the Q network.

It is hard to find the $\max_{a'} \boldsymbol{Q_\theta}(s',a')$ since $\boldsymbol{Q_\theta}(s',a')$ is (most of the case) non-convex.

That is, we cannot perform Q-learning update.

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- Continuous policy function is required.
- Let’s use Actor-critic scheme.
  - Critic: Q-network
  - Actor: **Deterministic** policy
    - Why **Deterministic ?** It has several advantages over stochastic policy (Will be explained in the following slides)

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- Recall Bellman expectation equation

$E$ : Environment
$\pi$: Target policy

$$Q^{\pi}(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E}\left[r(s_t, a_t) + \gamma\, \mathbb{E}_{a_{t+1} \sim \pi}\left[Q^{\pi}(s_{t+1}, a_{t+1})\right]\right]$$

That is, **Environment** and **Policy** determine $Q^{\pi}(s_t, a_t)$

**If** $\pi$ is deterministic, (we will use $\mu(s)$ to denote deterministic policy hereafter)

$$Q^{\mu}(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E}\left[r(s_t, a_t) + \gamma Q^{\mu}\left(s_{t+1}, \mu(s_{t+1})\right)\right]$$

That is, **Environment** solely determines $Q^{\mu}(s_t, a_t)$

**Advantage 1)** This selection enables us to train the model in "off-policy" fashion.

**Advantage 2)** No need to estimate Inner expectation values → Less variance

*Follows the notation of the DDPG paper

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- Remaining question:
  - Can we actually train continuous deterministic policy?
  - If yes, what is the loss function for continuous deterministic policy?
- Answer is “yes we can”

**Deterministic Policy Gradient Algorithms**

**David Silver** DAVID@DEEPMIND.COM
DeepMind Technologies, London, UK
**Guy Lever** GUY.LEVER@UCL.AC.UK
University College London, UK
**Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller** *@DEEPMIND.COM
DeepMind Technologies, London, UK

**Abstract**

In this paper we consider *deterministic* policy gradient algorithms for reinforcement learning with continuous actions. The deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function. This simple form means that the deterministic policy gradient can be estimated much more efficiently than the usual stochastic policy gradient. To ensure adequate exploration, we introduce an off-policy actor-critic algorithm that learns a deterministic target policy from an exploratory behaviour policy. We demonstrate that deterministic policy gradient algorithms can significantly outperform their stochastic counterparts in high-dimensional action spaces.

case, as policy variance tends to zero, of the stochastic policy gradient.

From a practical viewpoint, there is a crucial difference between the stochastic and deterministic policy gradients. In the stochastic case, the policy gradient integrates over both state and action spaces, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.

In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that our deterministic policy gradient algorithms continue to explore satisfactorily, we introduce an off-policy learning algorithm. The basic idea is to choose actions according to a stochastic behaviour policy (to ensure adequate exploration), but to learn about a deterministic target policy (exploiting the efficiency of the deterministic policy gradient). We use the

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- So far, the policy function $\pi(.|s)$ is always modeled as a probability distribution over actions $a$ given the current state $s$ and thus it is stochastic.
- Deterministic policy gradient (DPG) instead models the policy as a deterministic decision: $a = \mu(s)$.
- How can you calculate the gradient of the policy function when it outputs a single action?
- Let's define few notations to facilitate the discussion:
  - $\rho_0(s)$: The initial distribution over states
  - $\rho^\mu(s \to s', k)$: Starting from state $s$, the visitation probability density at state $s'$ after moving $k$ steps

    $\rho^\mu(s') = \int_S \sum_{k=1}^{\infty} \gamma^{k-1} \rho_0(s) \rho^\mu(s \to s', k) ds$ Discounted state distribution, defined as

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- The objective function to optimize for is listed as follows:

$$J(\theta) = \int_S \rho^{\mu}(s) Q\big(s, \mu_\theta(s)\big) ds$$

- Deterministic policy gradient theorem:

$$\nabla_\theta J(\theta) = \int_S \rho^{\mu}(s) \nabla_a Q^{\mu}(s,a) \nabla_\theta \mu_\theta(s) \Big|_{a=\mu_\theta(s)} ds$$

$$= \mathbb{E}_{s\sim\rho^{\mu}} \left[ \nabla_a Q^{\mu}(s,a) \nabla_\theta \mu_\theta(s) \Big|_{a=\mu_\theta(s)} \right]$$

- We can consider the deterministic policy as a special case of the stochastic one, when the probability distribution contains only one extreme non-zero value over one action.

- We expect the stochastic policy to require more samples as it integrates the data over the whole state and action space.

# 1. Deterministic Policy Gradient (Silver et al., 2014)

The cost of **stochastic** policy $J(\boldsymbol{\pi_\theta})$ is as follows:

$$J(\boldsymbol{\pi_\theta}) = \int_{s\in\mathcal{S}} \rho^{\pi_\theta}(s) \int_{a\in\mathcal{A}} \pi_\theta(s,a) r(s,a)\, da\, ds$$
$$= \mathbb{E}_{s\sim\rho^{\pi_\theta}, a\sim\pi_\theta}[r(s,a)]$$

The cost of **deterministic** policy $J(\boldsymbol{\mu_\theta})$ is as follows:

$$J(\boldsymbol{\mu_\theta}) = \int_{s\in\mathcal{S}} \rho^{\mu_\theta}(s)\, r\big(s, \mu_\theta(s)\big)\, ds$$
$$= \mathbb{E}_{s\sim\rho^{\pi_\theta}}\big[r\big(s, \mu_\theta(s)\big)\big]$$

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- The gradient of $J(\boldsymbol{\mu_\theta})$: (Theorem 1 of DPG paper)

$$\nabla_\theta J(\mu_\theta) = \int_{s \in \mathcal{S}} \rho^{\mu_\theta}(s)\, \nabla_\theta \mu_\theta(s)\, \nabla_a Q(s,a)\Big|_{a=\mu_\theta(s)}\, ds$$
$$= \mathbb{E}_{s \sim \rho^{\pi_\theta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q(s,a)\Big|_{a=\mu_\theta(s)}\right]$$

+ **known fact:** Deterministic policy gradient is the limiting case of stochastic policy gradient (Theorem 2 of DPG paper)

What we need to implement the algorithm:

1) A <u>deterministic</u> policy function that allows us to compute gradient w.r.t $\theta$ easily.
2) An action-value function $Q(s,a)$ that allows us to compute gradient w.r.t $a$.

In DDPG, both are neural networks.

* Proof of both theorem can be found from "Deterministic Policy Gradient Algorithms: Supplementary Material"

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- The deterministic policy gradient theorem can be plugged into common policy gradient frameworks.

$$J(\theta) = \int_S \rho^{\mu}(s) Q\big(s, \mu_\theta(s)\big) ds$$

- on-policy actor-critic algorithm with deterministic policy $a = \mu_\theta(s)$

$$\begin{aligned} \delta_t &= R_t + \gamma Q_w(s_{t+1}, a_{t+1}) - Q_w(s_t, a_t) \\ w_{t+1} &= w_t + \alpha_w \delta_t \nabla_w Q_w(s_t, a_t) \\ \theta_{t+1} &= \theta_t + \alpha_\theta \nabla_a Q_w(s_t, a_t) \nabla_\theta \mu_\theta(s) \Big|_{a_t = \mu_\theta(s_t)} \end{aligned}$$

- However, unless there is sufficient noise in the environment, it is very hard to guarantee enough exploration due to the determinacy of the policy.
  - We can either add noise into the policy (ironically this makes it nondeterministic!) or
  - learn it off-policy way by following a different stochastic behavior policy to collect samples.

# 1. Deterministic Policy Gradient (Silver et al., 2014)

- We want to have continuous policy.
  - Vanilla (Deep) Q-learning is not eligible for continuous action spaces.
  - Actor-critic is natural
- By selecting deterministic continuous policy,
  - We can train the critic in “off-policy” fashion.

- Loss functions:
  - Use Bellman error to train critic
  - Use deterministic Policy gradient to train actor

# Exploration problem revisit

- Since we choose to use deterministic policy, we confront "exploration problem" again.

- Other (deep) RL methods utilize exploration methods of their own.
  - MC with exploring starts
  - DQN with $\epsilon$-greedy
  - Entropy bonus on PG methods
  - Soft value functions.
  - …

# Exploration with Ornstein-Uhlenbeck process

Recall that we can train the models in “off-policy” fashion

i.e., RL agent behaves arbitrarily. still the agent can learn from the arbitrary experience!

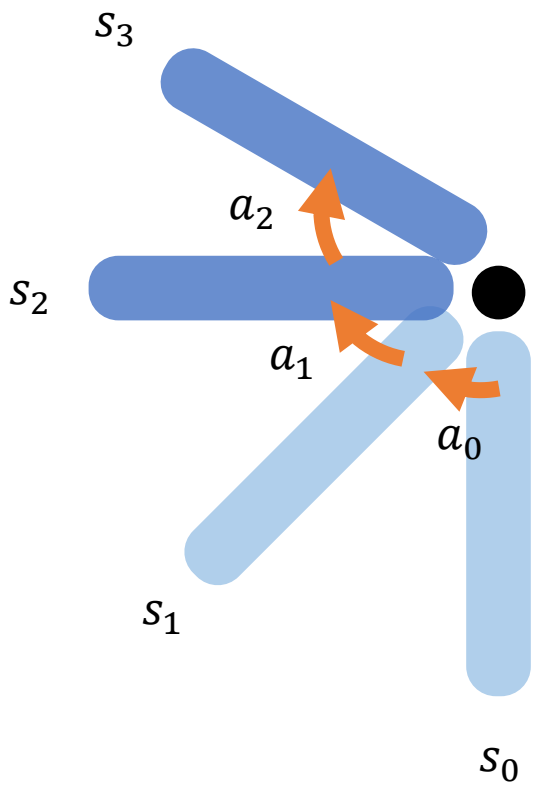
Q) Then what about using random behavioral policy such as $U \sim [a_{min}, a_{max}]$**?**

A) The random behavioral policy would not be the good choice.

Possibly because of following problems,

1. random action will force to optimize policy parameters on where the value estimate $Q$ is not accurate.
2. Doing (random) action but “consistently along the episode” is helpful for exploring unseen states.
3. Practically, random jittering would break the physical system.

# Exploration with Ornstein-Uhlenbeck process




- If we use 'random' behavioral policy, then can RL agent find the "intuitive optimal action"?
- It might find but after consumes a lot of samples.

**(Inverted pendulum problem)**

Objective: To make the pole upright and stay in upright position

"Intuitive optimal actions"

(1) Push the pole in consistent direction until it near to the upright position

(2) When the pole is near to the upright position, alternating the forcing directions.

# Exploration with Ornstein-Uhlenbeck process

- Ornstein-Uhlenbeck process generates random variables those are temporally correlated.
- Ornstein-Uhlenbeck process $x_t$ is defined by the following stochastic differential equation:

$$dx_t = -\theta x_t \, dt + \sigma \, dW_t$$

where $\theta > 0$ and $\sigma > 0$ are parameters and $W_t$ follows Wiener process.

- Langevin equation form of Ornstein-Uhlenbeck process:

$$\frac{dx_t}{dt} = -\theta x_t + \sigma \eta(t)$$

Where $\eta(t)$ is white noise.

# Deep Deterministic Policy Gradient (DDPG) (Lillicrap, et al., 2015)

- DDPG is an off-policy actor-critic model
  - Critic loss: $\frac{1}{N}\sum_i^N (Q^\mu(s_i, a_i) - r_i + \gamma Q^\mu\big(s_{i+1}, \mu(s_{i+1})\big)^2$
  - Actor loss gradient : $\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^{\pi_\theta}}\left[\nabla_\theta \mu_\theta(s) \nabla_a Q(s,a)|_{a=\mu_\theta(s)}\right]$
  - Use behavioral policy $\mu'(s_t) = \mu_\theta(s_t) + \epsilon$
    - The author choose $\epsilon$ to follow Ornstein-Uhlenbeck process.

  + Replay buffer

  + Target network for both of policy and critic.

# Deep Deterministic Policy Gradient (DDPG) (Lillicrap, et al., 2015)

**Algorithm 1** DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights $\theta^Q$ and $\theta^\mu$.
Initialize target network $Q'$ and $\mu'$ with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer $R$
**for** episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state $s_1$
  **for** t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action $a_t$ and observe reward $r_t$ and observe new state $s_{t+1}$
    Store transition $(s_t, a_t, r_t, s_{t+1})$ in $R$
    Sample a random minibatch of $N$ transitions $(s_i, a_i, r_i, s_{i+1})$ from $R$
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  **end for**
**end for**

for every transition

Gradually changing target networks

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim\rho^\beta}\left[\nabla_a Q^\mu(s,a)\nabla_\theta\mu_\theta(s)\Big|_{a=\mu_\theta(s)}\right]$$

Approximate it using minibatch samples

# Implementation Trick

<Typical implementation of DDPG using pytorch>

```python
import torch.nn as nn

class DDPG(nn.Module):

  def __init__(self, qnet, actor, qnet_opt, actor_opt,
               *args, **kwargs):
    self.qnet = qnet
    self.actor = actor
    self.qnet_opt = qnet_opt
    self.actor_opt = actor_opt

  def fit():
    "Some code after"

    # compute actor loss
    pi_loss = - self.qnet(state, self.actor(state))
    pi_loss = pi_loss.mean()
    pi_loss.backward()
    self.actor_opt.step()
```

Actor loss : $\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^{\pi_\theta}}\left[\nabla_\theta \mu_\theta(s) \nabla_a Q(s,a)|_{a=\mu_\theta(s)}\right]$

pi_loss : $Q_\phi\left(s, \mu_\theta(s)\right)$

pi_loss.backward() computes following partial gradients:

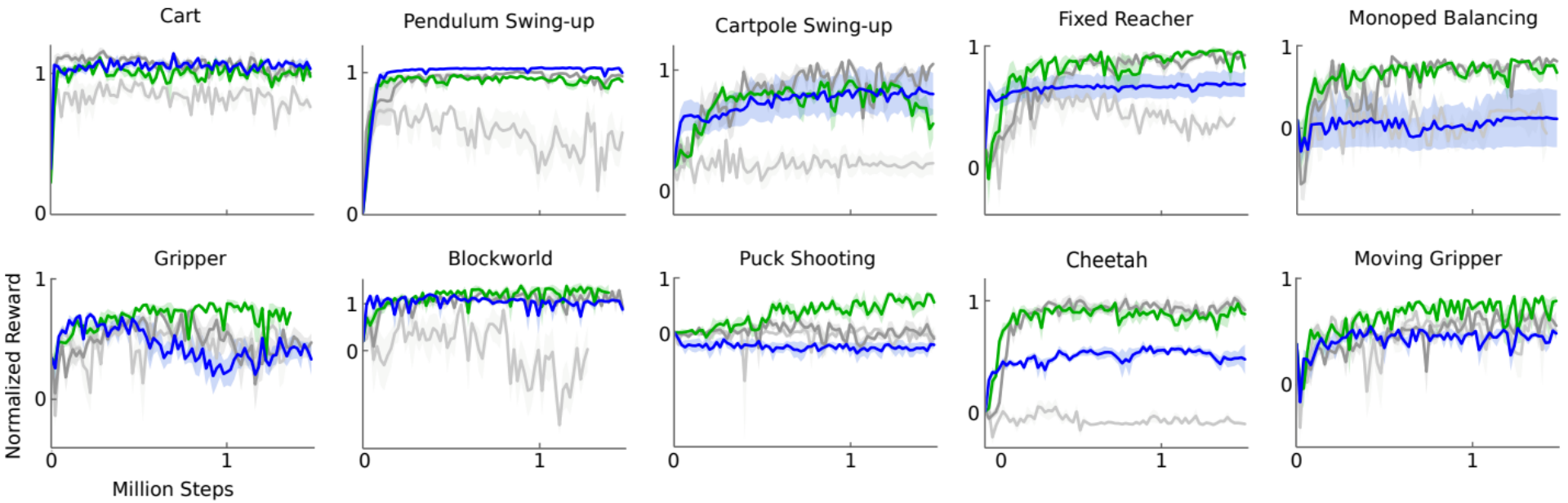
$$\frac{\partial Q_\phi\left(s,\mu_\theta(s)\right)}{\partial \theta}$$

$$\begin{aligned}\frac{\partial Q_\phi\left(s,\mu_\theta(s)\right)}{\partial \theta} &= \frac{\partial Q_\phi\left(s,\mu_\theta(s)\right)}{\partial \mu_\theta(s)}\frac{\partial \mu_\theta(s)}{\partial \theta} \\ &= \frac{\partial Q_\phi\left(s,\mu_\theta(s)\right)}{\partial a}\frac{\partial \mu_\theta(s)}{\partial \theta} \\ &= \nabla_\theta \mu_\theta(s)\, \nabla_a Q(s,a)\Big|_{a=\mu_\theta(s)}\end{aligned}$$

# Experiments

Continuous control on Mujoco environments

- DDPG **(Green)**
- DDPG w/o Target networks **(Light grey)**
- DDPG w/o BN* **(Dark grey)**
- DDPG from pixel inputs **(Blue)**




**Target networks matter a lot!**

*Batch normalization (BN) is a technique for boosting performance of neural network. DDPG trains in batch. Therefore DDPG can use BN.

# Experiments

**Performance comparison on input types**

The author trained two different DDPG models. The inputs for the models were *low-dimensional observation* and *pixel inputs*.

They refer the models as **DDPG-*lowd*** and **DDPG-*pix***.

They rewards are normalized so that they become 0 and 1 when the policy is as good as nearly random and planning, respectively.

| environment | $R_{av,lowd}$ | $R_{best,lowd}$ | $R_{av,pix}$ | $R_{best,pix}$ | $R_{av,cntrl}$ | $R_{best,cntrl}$ |
|---|---|---|---|---|---|---|
| blockworld1 | 1.156 | 1.511 | 0.466 | 1.299 | -0.080 | 1.260 |
| blockworld3da | 0.340 | 0.705 | 0.889 | 2.225 | -0.139 | 0.658 |
| canada | 0.303 | 1.735 | 0.176 | 0.688 | 0.125 | 1.157 |
| canada2d | 0.400 | 0.978 | -0.285 | 0.119 | -0.045 | 0.701 |
| cart | 0.938 | 1.336 | 1.096 | 1.258 | 0.343 | 1.216 |
| cartpole | 0.844 | 1.115 | 0.482 | 1.138 | 0.244 | 0.755 |
| cartpoleBalance | 0.951 | 1.000 | 0.335 | 0.996 | -0.468 | 0.528 |
| cartpoleParallelDouble | 0.549 | 0.900 | 0.188 | 0.323 | 0.197 | 0.572 |
| cartpoleSerialDouble | 0.272 | 0.719 | 0.195 | 0.642 | 0.143 | 0.701 |
| cartpoleSerialTriple | 0.736 | 0.946 | 0.412 | 0.427 | 0.583 | 0.942 |
| cheetah | 0.903 | 1.206 | 0.457 | 0.792 | -0.008 | 0.425 |
| fixedReacher | 0.849 | 1.021 | 0.693 | 0.981 | 0.259 | 0.927 |
| fixedReacherDouble | 0.924 | 0.996 | 0.872 | 0.943 | 0.290 | 0.995 |
| fixedReacherSingle | 0.954 | 1.000 | 0.827 | 0.995 | 0.620 | 0.999 |
| gripper | 0.655 | 0.972 | 0.406 | 0.790 | 0.461 | 0.816 |
| gripperRandom | 0.618 | 0.937 | 0.082 | 0.791 | 0.557 | 0.808 |
| hardCheetah | 1.311 | 1.990 | 1.204 | 1.431 | -0.031 | 1.411 |
| hopper | 0.676 | 0.936 | 0.112 | 0.924 | 0.078 | 0.917 |
| hyq | 0.416 | 0.722 | 0.234 | 0.672 | 0.198 | 0.618 |
| movingGripper | 0.474 | 0.936 | 0.480 | 0.644 | 0.416 | 0.805 |
| pendulum | 0.946 | 1.021 | 0.663 | 1.055 | 0.099 | 0.951 |
| reacher | 0.720 | 0.987 | 0.194 | 0.878 | 0.231 | 0.953 |
| reacher3daFixedTarget | 0.585 | 0.943 | 0.453 | 0.922 | 0.204 | 0.631 |
| reacher3daRandomTarget | 0.467 | 0.739 | 0.374 | 0.735 | -0.046 | 0.158 |
| reacherSingle | 0.981 | 1.102 | 1.000 | 1.083 | 1.010 | 1.083 |
| walker2d | 0.705 | 1.573 | 0.944 | 1.476 | 0.393 | 1.397 |
| torcs | -393.385 | 1840.036 | -401.911 | 1876.284 | -911.034 | 1961.600 |

# Multi-agent DDPG (MADDPG) (Lowe et al., 2017)

- Multi-agent DDPG (MADDPG) extends DDPG to an environment where multiple agents are coordinating to complete tasks with only local information.
- In the viewpoint of one agent, the environment is non-stationary as policies of other agents are quickly upgraded and remain unknown.
  - ➢ MADDPG is an actor-critic model redesigned particularly for handling such a non-stationary environment and interactions between agents.
- The problem can be formalized in the multi-agent version of MDP, also known as Markov games

$\mathcal{N}$: set of agents
$\mathcal{S}$: set of joint states
$\mathcal{A}_i$: set of possible action for agent $i$; $\mathcal{A} = \mathcal{A}_1 \times \cdots \times \mathcal{A}_N$ set of joint action
$\mathcal{O}_i$: set of observation for agent $i$; $\mathcal{O} = \mathcal{O}_1 \times \cdots \times \mathcal{O}_N$: set of joint observation
$\mathcal{T}: \mathcal{S} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_N \longmapsto \mathcal{S}$ transition function
$\pi_{\theta_i}: \mathcal{O}_i \times \mathcal{A}_i \longmapsto [0,1]$ stochastic policy of agent $i$
$\mu_{\theta_i}: \mathcal{O}_i \longmapsto \mathcal{A}_i$ deterministic policy of agent $i$

# Multi-agent DDPG (MADDPG) (Lowe et al., 2017)

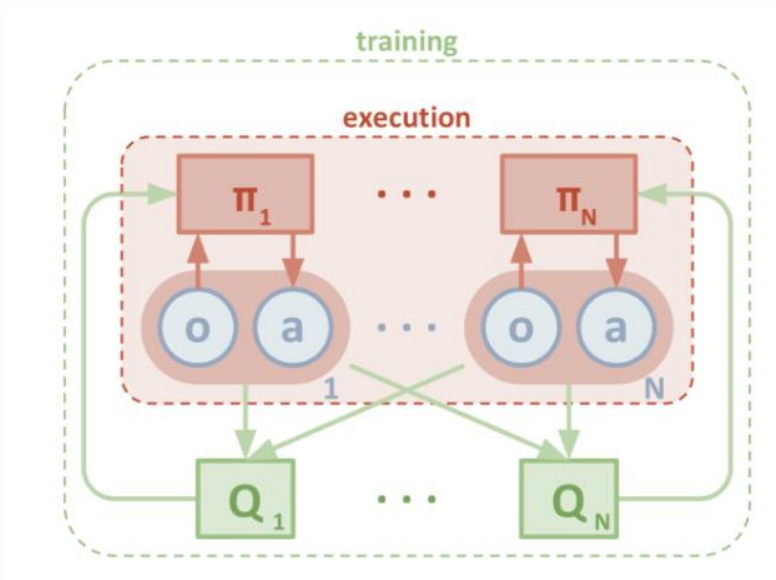
- The critic in MADDPG learns a centralized action-value function for agent $i$

$$Q_i^{\vec{\mu}}(\vec{o}, a_1, \dots, a_N)$$

  - ✓ $\vec{\mu} = \mu_1, \dots, \mu_N$ :joint policies
  - ✓ $\vec{o} = o_1, \dots, o_N$ :joint observations
  - ✓ $\vec{a} = a_1, \dots, a_N$ :joint actions

- Each $Q_i^{\vec{\mu}}$ is **learned separately** for $i = 1, \dots, N$ and therefore **multiple agents can have arbitrary reward structures**, including conflicting rewards in a competitive setting.

- Meanwhile multiple actors, one for each agent, are exploring and upgrading the policy parameters $\theta_i$ on their own.



# Multi-agent DDPG (MADDPG) (Lowe et al., 2017)

- **Actor update:**

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{(\vec{o},a)\sim D}\left[\nabla_{a_i} Q_i^{\vec{\mu}}(\vec{o}, a_1, \dots, a_N)\nabla_{\theta_i}\mu_{\theta_i}(o_i)\Big|_{a_i=\mu_{\theta_i}(o_i)}\right]$$

  - ✓ $D$ is the memory buffer for experience reply, containing multiple episode samples $(\vec{o}, a_1, \dots, a_N, r_1, \dots, r_N, \vec{o}')$: given current observation $\vec{o}$, agents take joint actions $a_1, \dots, a_N$ and get rewards $r_1, \dots, r_N$, leading to the new observation $\vec{o}'$

- **Critic update:**

$$\mathcal{L}(\theta_i) = \mathbb{E}_{(\vec{o},a_1,\dots,a_N,r_1,\dots,r_N,\vec{o}')\sim D}\left[\left(Q_i^{\vec{\mu}}(\vec{o}, a_1, \dots, a_N) - y\right)^2\right]$$

$$\text{where } y = r_i + \gamma\, Q_i^{\vec{\mu}'}(\vec{o}', a_1', \dots, a_N')\Big|_{a_i'=\mu_{\theta_i}'(o_i)}$$

  - ✓ $\vec{\mu}'$ are the target policies with delayed softly-updated parameters.
  - ✓ For each agent need to have $\vec{\mu} = \mu_1, \dots, \mu_N$ to compute the joint actions at the next observation $\vec{o}'$
  - ✓ Each agent learns other agents policy its own during the learning process.
  - ✓ Using the approximated policies, MADDPG still can learn efficiently although the inferred policies might not be accurate.

# Multi-agent DDPG (MADDPG) (Lowe et al., 2017)

- To mitigate the high variance triggered by the interaction between competing or collaborating agents in the environment, MADDPG proposed one more element - policy ensembles:

  - ✓ Train K policies for one agent;
  - ✓ Pick a random policy for episode rollouts;
  - ✓ Take an ensemble of these K policies to do gradient update.

- In summary, MADDPG added three additional ingredients on top of DDPG to make it adapt to the multi-agent environment:

  - ✓ Centralized critic + decentralized actors;
  - ✓ Actors are able to use estimated policies of other agents for learning;
  - ✓ Policy ensembling is good for reducing variance.

# Trust region policy optimization (TRPO) (Schulman, et al., 2015)

- To improve training stability, we should avoid parameter updates that change the policy too much at one step.
- Trust region policy optimization (TRPO) carries out this idea by enforcing a KL divergence constraint on the size of policy update at each iteration.

- **If off policy**, the objective function measures the total advantage over the state visitation distribution and actions, while the rollout is following a different behavior policy $\beta(a|s)$:

$$
\begin{aligned}
J(\theta) &= \sum_{s \in \mathcal{S}} \rho^{\pi_{\theta_{\text{old}}}} \sum_{a \in \mathcal{A}} \pi_\theta(a|s) \hat{A}_{\theta_{\text{old}}}(s,a) \\
&= \sum_{s \in \mathcal{S}} \rho^{\pi_{\theta_{\text{old}}}} \sum_{a \in \mathcal{A}} \beta(a|s) \frac{\pi_\theta(a|s)}{\beta(a|s)} \hat{A}_{\theta_{\text{old}}}(s,a) \\
&= \mathbb{E}_{s \sim \rho^{\pi_{\theta_{\text{old}}}}, a \sim \beta} \left[ \frac{\pi_\theta(a|s)}{\beta(a|s)} \hat{A}_{\theta_{\text{old}}}(s,a) \right]
\end{aligned}
$$

✓ $\theta_{\text{old}}$ is the policy parameters before the update and thus known to us. $\rho^{\pi_{\theta_{\text{old}}}}$ is defined similary.
✓ $\beta(a|s)$ is the behavior policy for collecting trajectories

# Trust region policy optimization (TRPO) (Schulman, et al., 2015)

- **If on policy**, the behavior policy is $\pi_\theta(a|s)$:

$$J(\theta) = \mathbb{E}_{s\sim\rho^{\pi_{\theta_{\text{old}}}}, a\sim\beta}\left[\frac{\pi_\theta(a|s)}{\beta(a|s)}\hat{A}_{\theta_{\text{old}}}(s,a)\right]$$

$$= \mathbb{E}_{s\sim\rho^{\pi_{\theta_{\text{old}}}}, a\sim\pi_{\theta_{\text{old}}}}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}\hat{A}_{\theta_{\text{old}}}(s,a)\right]$$

- TRPO aims to maximize the objective function $J(\theta)$ subject to, trust region constraint which enforces the distance between old and new policies measured by KL-divergence to be small enough, within a parameter $\delta$

$$\mathbb{E}_{s\sim\rho^{\pi_{\theta_{\text{old}}}}}\left[D_{KL}\left(\pi_{\theta_{\text{old}}}(\cdot|s)\|\pi_\theta(\cdot|s)\right)\right] \le \delta$$

- In this way, the old and new policies would not diverge too much when this hard constraint is met.
  - ➢ While still, TRPO can guarantee a monotonic improvement over policy iteration (why?)

# Proximal Policy Optimization (PPO) (Schulman, et al., 2017)

- Given that TRPO is relatively complicated and we still want to implement a similar constraint, proximal policy optimization (PPO) simplifies it by using a clipped surrogate objective while retaining similar performance.
- Denote the probability ratio between old and new policies as:

$$r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$$

- Then, the objective function of TRPO (on policy) becomes:

$$J^{\text{TRPO}}(\theta) = \mathbb{E}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}\hat{A}_{\theta_{\text{old}}}(s,a)\right] = \mathbb{E}\left[r(\theta)\hat{A}_{\theta_{\text{old}}}(s,a)\right]$$

- Without a limitation on the distance between $\theta_{\text{old}}$ and $\theta$, to maximize $J^{\text{TRPO}}(\theta)$ would lead to instability with extremely large parameter updates and big policy ratios.
- PPO imposes the constraint by forcing $r(\theta)$ to be within a small interval around 1, precisely $[1-\epsilon, 1+\epsilon]$

$$J^{\text{CLIP}}(\theta) = \mathbb{E}\left[\min\left(r(\theta)\hat{A}_{\theta_{\text{old}}}(s,a), \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_{\theta_{\text{old}}}(s,a)\right)\right]$$

  - ✓ The function $\text{clip}\,(r(\theta), 1-\epsilon, 1+\epsilon)$ clips the ratio $r(\theta)$ within $[1-\epsilon, 1+\epsilon]$.

# Proximal Policy Optimization (PPO) (Schulman, et al., 2017)

- When applying PPO on the network architecture with shared parameters for both policy (actor) and value (critic) functions, the objective function is augmented composed of
  - ✓ the clipped reward
  - ✓ an error term on the value estimation
  - ✓ an entropy term to encourage sufficient exploration.

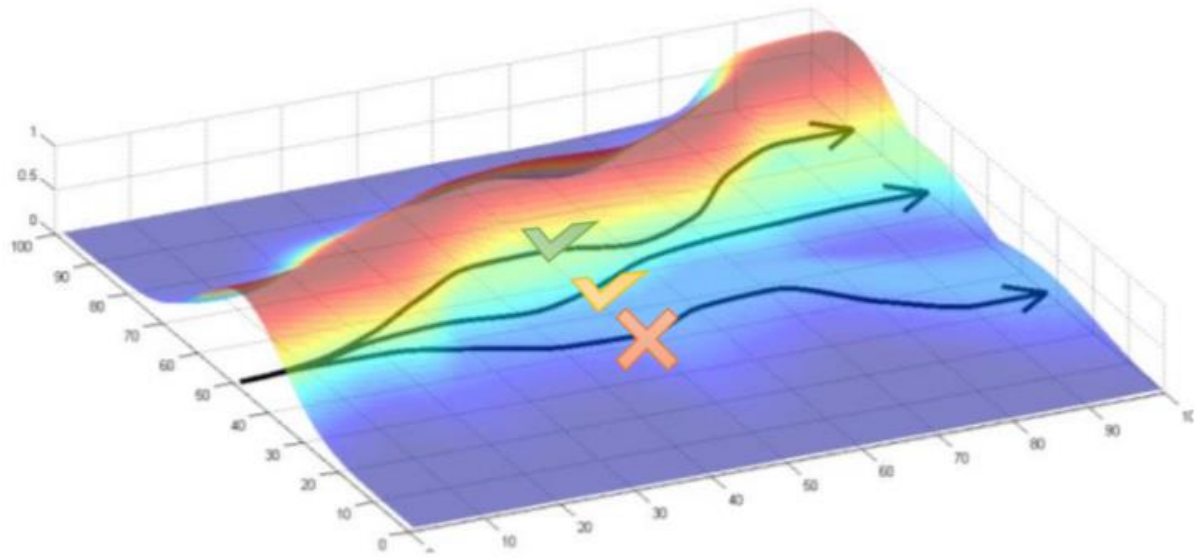
$$J^{\text{CLIP}'}(\theta) = \mathbb{E}\left[J^{\text{CLIP}}(\theta) - c_1\left(V_\theta(s) - V_{target}\right)^2 + c_2 H\left(s, \pi_\theta(\cdot)\right)\right]$$

$c_1$ and $c_2$ are two hyperparameters constants.

# Policy Gradient Algorithm From Different Angle

# Policy Gradient Algorithm From Different Angle

- We derived Policy Gradient Theorem using the concept of state value function to employ recursive formula on the basis of dynamic programming (Bellman equation)
- Not let's derive similar result without using DP approach but using ***episodic*** approach where agent engages in multiple trajectories in its environment



# Policy Gradient Algorithm From Different Angle

- Let's define a trajectory $\tau$ of length $T$ as

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, s_T)$$

  - ✓ $s_0$ comes from the starting distribution of states
  - ✓ $a_i \sim \pi_\theta(a_i|s_i)$ with $\pi_\theta$ parameterized ***policy***
  - ✓ $s_i \sim P(s_i|s_{i-1}, a_{i-1})$ with $P$ dynamic model describing how ***environment*** changes

- The probability of trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, s_T)$ can be expressed as

$$p(\tau) = \mu(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$$

- We can compute

$$\begin{aligned}
\nabla_\theta \log p(\tau) &= \nabla_\theta \log \left( \mu(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t) \right) \\
&= \nabla_\theta \left[ \log \mu(s_0) + \sum_{t=0}^{T-1} (\log \pi_\theta(a_t|s_t) + \log P(s_{t+1}|s_t, a_t)) \right] \\
&= \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)
\end{aligned}$$

# Policy Gradient Algorithm From Different Angle

- We want to maximize

$$\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} r_t\right] = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)] \qquad \because \text{we know } \tau \text{ is influenced by } \pi_\theta$$

- The gradient of objective is computed as

$$\begin{aligned}
\nabla_\theta \mathbb{E}_{\tau\sim p_\theta}[R(\tau)] &= \nabla_\theta \int p_\theta(\tau) R(\tau) d\tau \\
&= \int \frac{p_\theta(\tau)}{p_\theta(\tau)} \nabla_\theta p_\theta(\tau) R(\tau) d\tau \\
&= \int p_\theta(\tau) \nabla_\theta \ln p_\theta(\tau)\, R(\tau) d\tau \\
&= \mathbb{E}_{\tau\sim p_\theta}[\nabla_\theta \ln p_\theta(\tau)\, R(\tau)] \\
&= \mathbb{E}_{\tau\sim \pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\, R(\tau)\right] \\
&= \mathbb{E}_{\tau\sim \pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \left(\sum_{t=0}^{T-1} r_t\right)\right]
\end{aligned}$$

$$\because \nabla_\theta \log p(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

# Policy Gradient Algorithm From Different Angle

- We have

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\left(\sum_{t=0}^{T-1} r_t\right)\right]$$

  - ✓ The expectation is with respect to the policy function, so we can think $\tau\sim\pi_\theta$

- We need trajectories to get an empirical expectation, which estimates the actual expectation. The naïve way is to run the agent on batch of episodes

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}_{\tau\in\mathcal{T}}[R(\tau)]$$

  - ✓ where $\mathcal{T}$ is a set of trajectories sampled

$$\nabla_\theta \mathbb{E}_{\tau\in\mathcal{T}}[R(\tau)] \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right)\left(\sum_{t=0}^{T-1} r_t^{(i)}\right)$$

  - ✓ $\tau^{(i)} = \left(s_0^{(i)}, a_0^{(i)}, r_0^{(i)}, s_1^{(i)}, a_1^{(i)}, r_1^{(i)}, \ldots, s_{T-1}^{(i)}, a_{T-1}^{(i)}, r_{T-1}^{(i)}, s_T^{(i)}\right)$ is the $i$-th trajectory.
  - ✓ This approach will be too slow and unreliable due to high variance on the gradient estimate
  - ✓ We need various ***variance reduction*** techniques

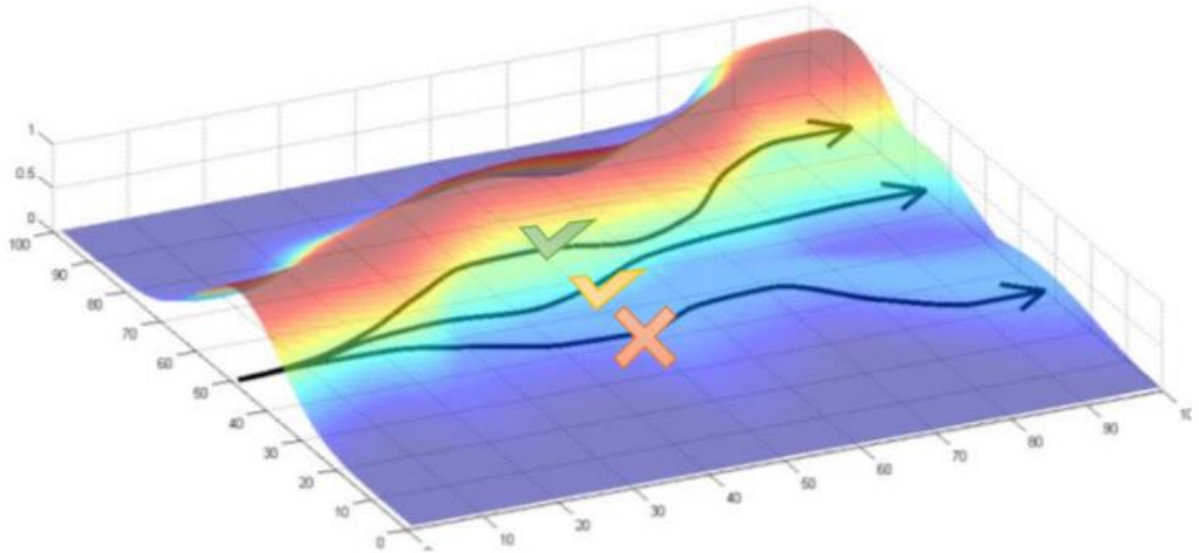
# Interpreting Policy Gradient

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\left(\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t|s_t)\right)\left(\sum_{t=0}^{T-1} r_t\right)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right)\right)\left(\sum_{t=0}^{T-1} r_t^{(i)}\right)$$

probability of trajectory

Reward of trajectory

$\nabla_\theta J_{ML}(\theta)$



- Increase the likelihood for the trajectory with large accumulated reward
- Decrease the likelihood for the trajectories with small accumulated reward

# Variance Reduction (Imposing Causality)

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\left(\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t|s_t)\right)\left(\sum_{t=0}^{T-1} r_t\right)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right)\right)\left(\sum_{t=0}^{T-1} r_t^{(i)}\right)$$

- Apply causality: policy at time $t'$ cannot affect reward at time $t$ when $t < t'$

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right)\right)\left(\sum_{t=0}^{T-1} r_t^{(i)}\right)$$

$$= \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right)\sum_{t'=t}^{T-1} r_{t'}^{(i)}$$

$$= \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right)\hat{Q}_t^{(i)}$$

"reward to go"

$$(f_0 + f_1 + \cdots + f_t + \cdots f_T)(f_0 + f_1 + \cdots + f_t + \cdots f_T)$$
$$= f_0(f_0 + f_1 + \cdots + f_t + \cdots f_T) +$$
$$f_1(f_0 + f_1 + \cdots + f_t + \cdots f_T) +$$
$$\vdots$$
$$f_t(f_0 + f_1 + \cdots + f_t + \cdots f_T) +$$
$$\vdots$$
$$f_T(f_0 + f_1 + \cdots + f_t + \cdots f_T) +$$

# Variance Reduction (Subtracting Baseline)

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right) \sum_{t'=t}^{T-1} r_{t'}^{(i)}\right] \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta\left(a_t^{(i)}\middle|s_t^{(i)}\right) \sum_{t'=t}^{T-1} r_{t'}^{(i)}$$

- Subtract baseline term to reduce variance

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t)\right)\right]$$

- We need to verify the things:
  1) Inserting baseline does not make the gradient estimate biased
  2) Baseline reduce actually variance
  - ➢ Staying unbiased and reducing variance is always good!

# Variance Reduction (Subtracting Baseline)

**(1) Inserting baseline does not make the gradient estimate biased**

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta \log\pi_\theta(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t)\right)\right]$$

$$= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta \log\pi_\theta(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'}\right) - \sum_{t=0}^{T-1}\nabla_\theta \log\pi_\theta(a_t|s_t)\, b(s_t)\right]$$

- All we need to show is that for any single time $t$, the gradient of $\log\pi_\theta(a_t|s_t)$ multiplied with $b(s_t)$ is zero:

$$\begin{aligned}
\mathbb{E}_{\tau\sim\pi_\theta}[\nabla_\theta \log\pi_\theta(a_t|s_t)\, b(s_t)] &= \mathbb{E}_{s_{0:t},a_{0:t-1}}\left[\mathbb{E}_{s_{t+1:T},a_{t:T-1}}[\nabla_\theta \log\pi_\theta(a_t|s_t)\, b(s_t)]\right] \\
&= \mathbb{E}_{s_{0:t},a_{0:t-1}}\left[b(s_t)\cdot\mathbb{E}_{s_{t+1:T},a_{t:T-1}}[\nabla_\theta \log\pi_\theta(a_t|s_t)]\right] \\
&= \mathbb{E}_{s_{0:t},a_{0:t-1}}\left[b(s_t)\cdot\mathbb{E}_{a_t}[\nabla_\theta \log\pi_\theta(a_t|s_t)]\right] \\
&= \mathbb{E}_{s_{0:t},a_{0:t-1}}[b(s_t)\cdot 0] = 0
\end{aligned}$$

$$\because \mathbb{E}_{a_t}[\nabla_\theta \log\pi_\theta(a_t|s_t)] = \int \frac{\nabla_\theta\pi_\theta(a_t|s_t)}{\pi_\theta(a_t|s_t)}\pi_\theta(a_t|s_t)da_t = \nabla_\theta\int \pi_\theta(a_t|s_t)\, da_t = \nabla_\theta\cdot 1 = 0$$

# Variance Reduction (Subtracting Baseline)

**(1) Inserting Baseline reduce actually variance**

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t)\right)\right]$$

$$= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'}\right) - \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\, b(s_t)\right]$$

- Why subtracting baseline reduces variance?

$$\mathrm{Var}\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\,(R_t(\tau) - b(s_t))\right) \approx \sum_{t=0}^{T-1} \mathbb{E}_{\tau\sim\pi_\theta}\left[\left(\nabla_\theta \log \pi_\theta(a_t|s_t)\,(R_t(\tau) - b(s_t))\right)^2\right] \qquad \mathrm{Var}\left(\sum_{i=1} X_i\right) \approx \sum_{i=1} \mathrm{Var}(X_i)$$

$$\left(R_t(\tau) = \sum_{t'=t}^{T-1} r_{t'}\right)$$

$$\approx \sum_{t=0}^{T-1} \mathbb{E}_{\tau\sim\pi_\theta}\left[(\nabla_\theta \log \pi_\theta(a_t|s_t))^2\right] \mathbb{E}_{\tau\sim\pi_\theta}\left[\left((R_t(\tau) - b(s_t))^2\right)\right] \qquad E[AB] = E[A]E[B]$$

- We can optimally choose $b(s_t)$ to minimize variance as

$$\min_{b(s_t)} \mathbb{E}_{\tau\sim\pi_\theta}\left[\left((R_t(\tau) - b(s_t))^2\right)\right]$$

✓ Which gives the least square solution $b(s_t) = \mathbb{E}_{\tau\sim\pi_\theta}[R_t(\tau)]$

➢ have to re-fit the baseline estimate each time to make it as close to the expected return

# Variance Reduction (Advantage Function)

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t)\right)\right]$$

$$= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\left(Q^\pi(s_t,a_t) - V^\pi(s_t)\right)\right]$$

$$= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\, A^\pi(s_t,a_t)\right]$$

$$= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\, \Psi_t\right]$$

- The problem of policy gradient is reduces to finding good estimates $\Psi_t$, a topic of ***Generalized Advantage Estimation*** (Schulman et al., 2016)

where $\Psi_t$ may be one of the following:

1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory.
2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action $a_t$.
3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula.
4. $Q^\pi(s_t, a_t)$: state-action value function.
5. $A^\pi(s_t, a_t)$: advantage function.
6. $r_t + V^\pi(s_{t+1}) - V^\pi(s_t)$: TD residual.